Rebuilding *BearcastMedia.com*

Architecture, content model, frontend modules, radio engineering, edge functions, build pipeline and automation of a student-media platform — from an inherited WordPress template to a headless, CMS-driven, edge-deployed system. Rewritten from the current source code, with the code itself quoted throughout.

VERSION 2.0 · SUPERSEDES V1.1 (AUG 2026)

AUTHORED BY

Cam Garrison
Web Director, Bearcast Media

SOURCE

notChewy1324/bearcastmedia
main · October 2026 snapshot

PUBLICATION

May 2026 · rev. Oct 2026
© 2026 Cam Garrison

Contents

§ 01	Abstract & Executive Summary	3
§ 02	Project Context & Inherited State	5
§ 03	System Architecture Overview	8
§ 04	Sanity CMS — Content Model & Studio	11
§ 05	Frontend Code & Page Architecture	20
§ 06	Loading Content from Sanity	26
§ 07	Search, the Command Palette & the Reader	34
§ 08	Radio: Player, Live Data & Requests	38
§ 09	Your Bearcast — Personal Stats Without Accounts	45
§ 10	BTV: YouTube as the Source of Truth	49
§ 11	Team, GBMs & the Interest Form	53
§ 12	Cloudflare Pages Functions	56
§ 13	Build, Deployment & the Offline Layer	61
§ 14	Automation, Quality Gates & Local Development	67
§ 15	Before & After — WordPress vs. the Current Build	71
§ 16	Performance, Operations & Known Limits	74
§ 17	Colophon, Glossary & Revision History	76

HOW TO READ THE CODE IN THIS PAPER

Every code listing is pulled automatically from the source tree at build time — nothing is retyped. The caption gives the file and the real line numbers, so any listing can be found in the repository in seconds. Highlighted lines mark the part the text is talking about; "… N lines omitted …" marks a deliberate cut.

§ 01 — ABSTRACT

Abstract & Executive Summary

> *This paper describes, from its source code, how BearcastMedia.com works today: a static, hand-authored front end on Cloudflare Pages, a custom-modeled Sanity Studio for content, Radio.co for live audio, and a thin layer of edge functions for the few things a static site cannot do on its own.*

Bearcast Media is the University of Cincinnati's student-run media organization, on the air since 2001 and celebrating its 25th anniversary in 2026. Its website replaced an inherited WordPress template with a site authored specifically for Bearcast. The founding decisions of that rebuild still hold: content lives in Sanity as structured, validated documents; the public site is plain HTML, CSS and JavaScript with no framework; pages are served from Cloudflare's edge and fetch their content in the browser; the radio streams from Radio.co.

What has changed since the first edition is everything built on top of that foundation. Between August and October 2026 the site gained a full radio product (live waveform and spectrum analysis of the actual stream, DJ-set detection, a live listener count, song requests that land in Slack, and **Your Bearcast** — personal listening recaps, streaks, tiers, badges and a year-end Wrapped, all stored only in the listener's browser); archive-wide search with typo tolerance and server-side full-text ranking; an article reader that can read stories aloud; automatic BTV listings from YouTube; real link previews for shared stories; a working interest form; a per-page build step; and GitHub Actions that check every pull request, test the live site weekly and back up the CMS weekly.

9	11,216	10,765	5	8 + 2	$0
HTML PAGES	LINES · SCRIPTS.JS	LINES · STYLES.CSS	EDGE FUNCTIONS	CMS DOC + OBJECT TYPES	MONTHLY HOSTING

1.1 Key decisions at a glance

CONCERN	DECISION	REASON
Rendering	Static HTML + client-side fetch	No framework to learn or upgrade; publishing needs no rebuild.
Content	Sanity (hosted, schema-defined)	Structured content, validation at the source, department-shaped editor, free tier.
Hosting	Cloudflare Pages	Free global edge, TLS, preview deploys, one-click rollback.
Server code	Five Pages Functions, only where needed	Secrets (Slack webhooks), link previews, sitemap, YouTube list, listener count. Everything else stays pure CDN (`_routes.json`).
Build	`build.sh` : per-page bundles, minify, version stamp	Each page downloads only its own CSS/JS; developers still edit one readable file of each.

CONCERN	DECISION	REASON
Audio	Native `<audio>` on the Radio.co stream + public JSON APIs	Full control of the player, lock-screen metadata, real-audio visualizers, auto-reconnect.
Listener data	`localStorage` only	Personal stats with no accounts and nothing sent to a server.
Security	Strict CSP with hash-pinned inline scripts, escaping helpers, schema validation	Defense in depth; enforced by CI on every pull request.
Departments	Hardcoded in code	They are routing primitives (pages, colours, nav), not editorial content.
Offline	Service worker (`sw.js`)	Visited pages work offline; live data is never cached.

1.2 What changed since version 1.1

AREA	V1.1 (AUGUST 2026)	V2.0 (THIS EDITION, CURRENT CODE)
Player	Radio.co iframe widget	Own player on a native `<audio>` element; Web Audio analysis; Media Session; reconnect with back-off (§08)
Build	“No build step”	`build.sh` + `split-assets.js` + pinned esbuild; per-page bundles; automatic asset versioning (§13)
Server	None	Pages Functions: song requests, interest form, listener count (D1), BTV list, link previews & sitemap (§12)
Fonts	Google Fonts CDN	Self-hosted woff2 files in `/fonts` (no third-party requests)
Theme	Light / dark	Auto / light / dark, following the OS live under Auto (§05)
CSP	`'unsafe-inline'` scripts allowed	Only two inline scripts, pinned by SHA-256 hash; no inline handlers anywhere (§05, Cybersecurity paper)
Search	⌘K over titles	Shared engine with typo tolerance, server-side full-text ranking, filters, page search boxes (§07)
BTV	Episodes typed into the Studio	Every YouTube upload appears automatically; Studio adds optional details (§10)
Forms	Stubs that sent nothing	Song requests and interest forms delivered to Slack (§08, §11)
Service worker	Precached every page; version bumped by hand	Caches pages as visited; cache version stamped automatically per deploy (§13)
Schema	9 document types incl. legacy `dept`	8 document types; `dept` removed; content migrations with rollbacks (§04)
Operations	Manual	PR checks, weekly health check, weekly backup, Dependabot (§14)

Correction: earlier editions gave the founding year as 1999. The site, the Studio defaults and the 25th-anniversary branding all use 2001, which this edition follows.

§ 02 — CONTEXT

Project Context & Inherited State

> *Bearcast Media runs five departments — Radio, Sports, BTV (television), Journalism and Music. The website is the only public surface fully under the organization's editorial control, and the only one searchable from outside.*

2.1 The inherited site

The site inherited at the start of this project was a WordPress installation running a commercial multi-purpose theme — the kind of catch-all template sold for small businesses, blogs and media alike — adopted years earlier by a cohort of students who needed a web presence quickly. That was a pragmatic choice, but its costs grew every year:

- **Plugin sprawl.** Features were spread across third-party plugins, each with its own update cadence and security obligations. The dashboard below shows pending updates, upsell banners and a radio-player add-on — the operational load a rotating student maintainer inherits.
- **Theme entanglement.** Layout, branding and display logic lived together in PHP templates, so a content change could break layout and vice versa.
- **Editorial mismatch.** WordPress models "posts" with categories and tags. A newsroom thinks in desks, shows, games, episodes and reviews.
- **Stated organizational preference.** The directors had asked for years to leave the template for a site authored for Bearcast. The rebuild was organization-pull, not technology-push.

INHERITED WORDPRESS ADMIN — BEFORE THE REBUILD

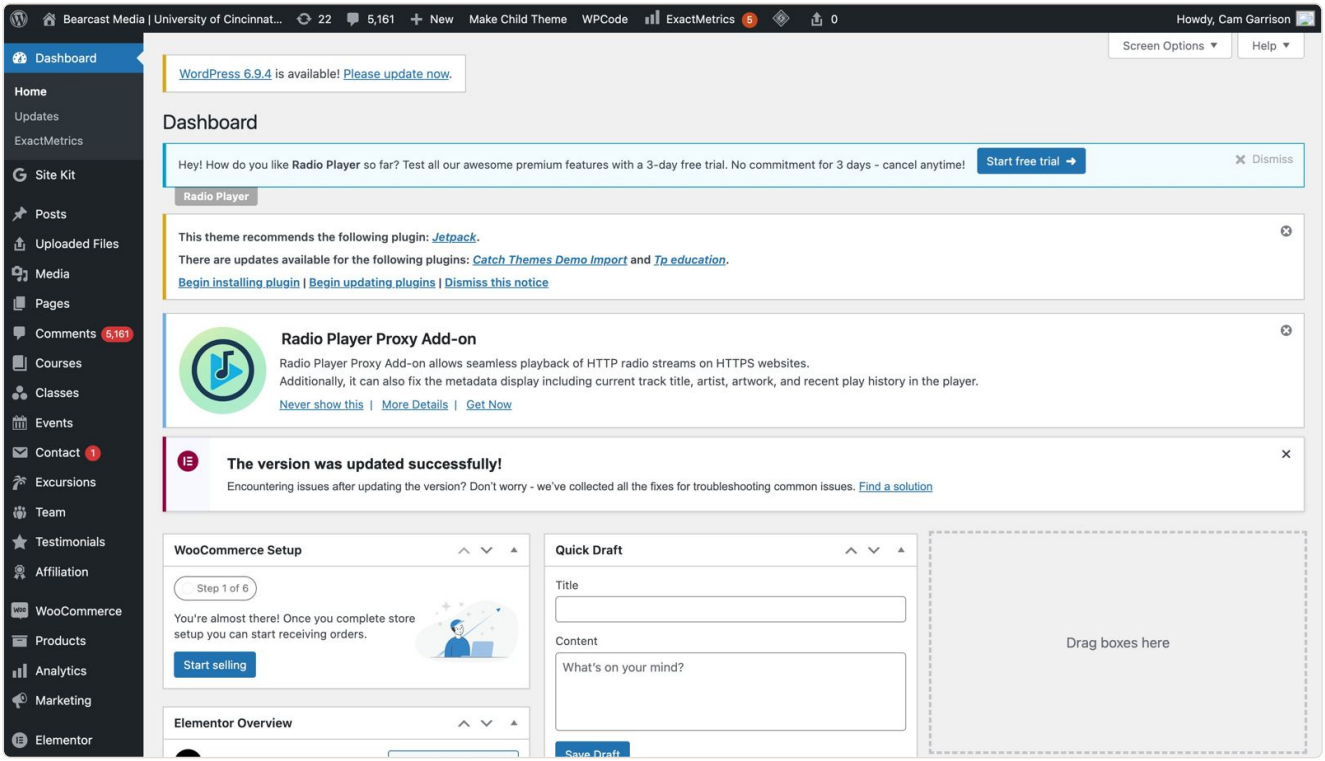


Figure 2.1 — The inherited WordPress dashboard: update notices, theme and plugin prompts, a premium radio-player add-on and a WooCommerce setup panel on a site that sells nothing. Screenshot from the original audit, May 2026.

2.2 Constraints on the rebuild

1. **Zero or near-zero running cost.** Student-org budgets are reset every year.
2. **Hand-off survivability.** The author will graduate (2028). The next maintainer may be a motivated student with web fundamentals, not a senior engineer.
3. **Editorial accessibility.** Editors rotate yearly and must publish without engineering help.
4. **Brand fidelity.** The independent, editorial Bearcast voice must come through, per department.
5. **No vendor traps.** The Sanity dataset is the durable artifact; the front end is replaceable.
6. **Run unattended.** The site must keep working for long stretches with no code changes — hence the automation in §14.

> DESIGN PRINCIPLE
>
> Treat the content layer as the durable asset and the rendering layer as disposable. A future maintainer should be able to throw away `scripts.js` and build a new front end against the same Sanity dataset without losing a single article, show or director profile.

2.3 Where the content came from

The existing WordPress archive was imported into Sanity, which is why many document IDs look like `wp-review-17767` or `wp-coverage-17803`. The import was mechanical, so in October 2026 a set of reviewed, reversible content migrations cleaned it up — real artists and albums for 183 reviews, consistent genres and sports, and 176 pieces moved to the content type they actually are (§4.8).

§ 03 — ARCHITECTURE

System Architecture Overview

> *Four independently hosted layers — content, edge, browser and audio — talk to each other only over public HTTPS. A small set of supporting services (YouTube, Apple's iTunes Search, Slack and GitHub) each do one job.*

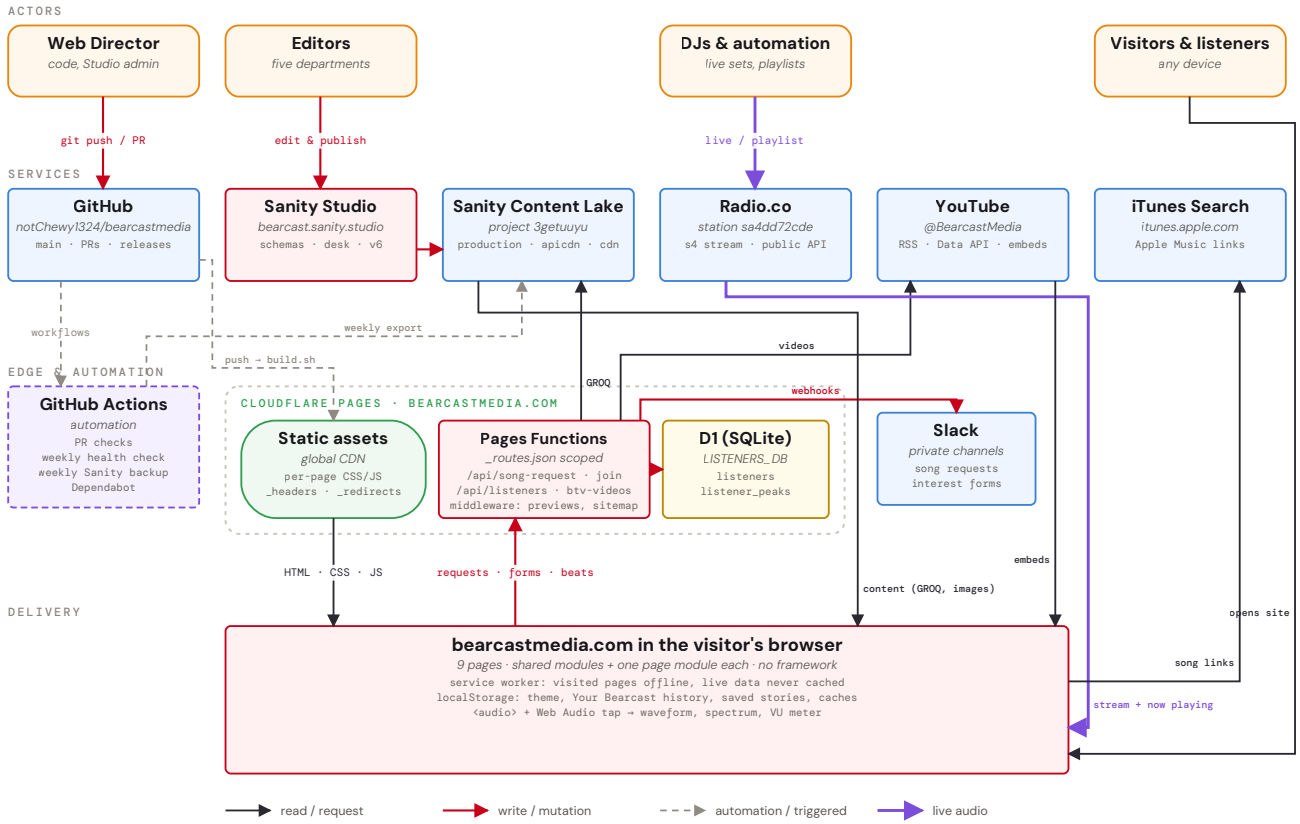


Figure 3.1 — The whole system. Solid arrows are requests the visitor's browser makes; red arrows are writes; purple is live audio; dashed arrows are automation. The companion *System Architecture Diagram* document draws each flow in detail.

3.1 The layers

LAYER	WHAT IT IS	WHERE IT LIVES
Content	Sanity Studio (editing UI) and the Content Lake (dataset `production`, project `3getuuyu`)	bearcast.sanity.studio · Sanity's cloud
Edge	Static files plus five Pages Functions and one D1 database	Cloudflare Pages project `bearcastmedia` (bearcastmedia.com, bearcastmedia.pages.dev)
Browser	One page module per page, shared modules, service worker; all rendering and personal data	The visitor's device

LAYER	WHAT IT IS	WHERE IT LIVES
Audio	Live stream and now-playing/history APIs	Radio.co station `sa4dd72cde` (`s4.radio.co` , `public.radio.co`)
Supporting	YouTube (BTV videos), iTunes Search (Apple Music links), Slack (requests & applications), GitHub (code, CI, backups)	Third-party services

3.2 Why static-first still wins

A server-rendered framework (Next.js, Astro) fetching Sanity at build time was considered and rejected, and the reasons have held up:

- **Build coupling.** Every publish would need a rebuild or a webhook-plus-revalidation system. Here, the next visitor sees a published change within about a minute (Sanity's API CDN cache).
- **Hand-off friction.** A successor can open `journalism.html` and the journalism block of `scripts.js` and read them top to bottom; there is no framework to learn first.
- **The real bottleneck** is one content fetch and font loading, which a framework would not remove.

The site did gain a build *step* (§13), but it is deliberately not a framework: it only splits and minifies the two files developers edit, stamps a version number and removes non-site files. Nothing is compiled, and the readable source still runs as-is when served locally.

3.3 Repository layout

README.MD · LINES 90-120 — FROM THE REPOSITORY README

```
bearcastmedia/
├── index.html … team.html     Site pages (one per section) + 404.html
├── scripts.js                 All site JavaScript — shared modules, then one block per page
├── styles.css                 All styles, page-scoped via body[data-page="…"]
│                              (build.sh turns these two into small per-page files on deploy)
├── sw.js                      Service worker (visited pages work offline; offline page otherwise)
├── imgs/                      Favicons and site images
├── fonts/                     Self-hosted web fonts (no Google Fonts requests)
├── functions/
│   ├── _middleware.js         Link previews + JSON-LD for shared stories, /sitemap.xml
│   └── api/
│       ├── song-request.js    Validates song requests → Slack webhook
│       ├── join.js            Join-page interest form → directors' private Slack channel
│       ├── listeners.js       Live listener count (Cloudflare D1)
│       └── btv-videos.js      Lists the BearcastMedia YouTube channel for the BTV page
├── _routes.json               Which paths run Functions (everything else is pure CDN)
├── robots.txt                 Points search engines at /sitemap.xml
├── _headers                   Security headers + CSP + caching (Cloudflare Pages)
├── _redirects                 404 handling
├── build.sh                   Cloudflare build step (per-page bundles + minify, asset versioning, strips
non-site files)
├── bearcast-studio/           Sanity Studio source (schemas, desk structure, config)
├── .github/
│   ├── workflows/             PR checks, weekly backup, weekly health check
│   ├── scripts/               PR checks, per-page bundle builder, health check
│   ├── dependabot.yml         Monthly GitHub Actions update PRs
│   └── pull_request_template.md
├── MAINTENANCE.md             Twice-a-year check-up, troubleshooting, where settings live
```

```
117 ├── CONTRIBUTING.md          How to contribute (code only — content lives in the CMS)
118 ├── SECURITY.md              How to report a vulnerability privately
119 ├── LICENSE                  MIT for the code; branding and content excluded
120 └── README.md                This file
```

The `scripts.js` layout is the key to the front end: shared modules first, then one self-contained block per page between `//#page <name>` and `//#endpage` markers. Each block starts by checking `<body data-page="…">` and returns immediately on any other page, so the single file is safe to load everywhere — and the build can cut it into per-page bundles without changing behaviour.

SCRIPTS.JS REGION	LINES	CONTENTS
Shared modules	1-2940	Config, action dispatcher, escaping helpers, focus trap, `BEARCAST_RADIO`, `BC_LISTENERS`, toasts, deep links, `sanityFetch`, `BC_SEARCH`, reader, dialog, `BC_BTV`, image URLs, Portable Text, loaders, site settings, `BC_SOCIAL`, theme, audio tap, `BC_ME`, footer year, cursor
`//#page home`	2946-3625	Hero player, mini player, waveform, now-on-air, hero extras
`//#page btv`	3631-3923	Episode grid, filters, modal player
`//#page radio`	3929-7707	Full player, visualizers, schedule, shows, tracks, Your Bearcast, requests
`//#page music`	7713-8456	Reviews, music journalism, music videos, reader
`//#page sports`	8462-9174	Coverage, sports journalism, scoreboard, reader
`//#page journalism`	9180-9685	Newspaper front, desks, reader
`//#page team`	9691-10041	Directors in seniority order, profiles
`//#page join`	10047-10376	GBM countdown and list, interest form
Shared (all pages)	10378-11216	Nav, back-to-top, ⌘K palette, reader keys, Konami code, offline notices, service-worker registration

§ 04 — CONTENT LAYER

Sanity CMS — Content Model & Studio

> *Sanity is the durable artifact of this project. If everything else were thrown away tomorrow, the dataset would still be a complete, structured record of every article, review, show, episode and person Bearcast has published.*

4.1 Project, dataset and API version

The Studio and the website point at the same project (`3getuuyu`) and dataset (`production`). These identifiers are public by design: they say *which* content store to read, not whether the caller may write. Write access is granted only through Sanity's own login; neither the website nor the repository holds a write token. The API version is pinned to `2024-01-01` in three places that must move together: the Studio config, the top of `scripts.js` and the health-check script.

BEARCAST-STUDIO/SANITY.CONFIG.TS · LINE 25 — STUDIO: THE PINNED API VERSION

```
25  const SANITY_API_VERSION = '2024-01-01'
```

SCRIPTS.JS · LINES 9-12 — WEBSITE: THE SAME VALUES

```
 9  window.BEARCAST_SANITY_ENABLED = true;
10  window.SANITY_PROJECT_ID = '3getuuyu';
11  window.SANITY_DATASET    = 'production';
12  window.SANITY_API_VERSION = '2024-01-01';
```

4.2 Modeling the newsroom, not the pages

The schema promotes structural differences to separate document types instead of WordPress-style categories on one "post" type. A game recap has a sport, matchup and final score; a review has an album, artist, score and verdict; a radio show has weekly time slots. Each type carries its own validation and its own Studio preview. Two nested types recur and are shared: `timeSlot` (inside radio shows) and `richBody` (the Portable Text body used by articles, reviews and coverage).

BEARCAST-STUDIO/SCHEMAS/INDEX.TS · LINES 23-37 — SCHEMAS/INDEX.TS — EVERY TYPE THE STUDIO KNOWS

```
23 export const schemaTypes = [
24   // Objects first (so they're available when documents reference them)
25   timeSlot,
26   richBody,
27
28   // Documents
29   siteSettings, // singleton — keep at top; see siteSettings.ts header for v3-compliant enforcement
30   article,
31   review,
32   coverage,
33   episode,
34   radioShow,
35   person,
36   gbm,
37 ]
```

TYPE	KIND	PURPOSE AND WHERE IT APPEARS
`siteSettings`	Singleton	Site-wide copy and social links. Today the website reads its **Social Media** links (footers, phone menu, Join, BTV "Subscribe", Sports "Follow"); the other fields are stored for future use.
`article`	Document	All written journalism. The `desk` field routes it: seven journalism desks → Journalism page; `MusicJournalism` → Music; `SportsJournalism` → Sports.
`review`	Document	Album / track reviews: score 0–10, verdict, artist, album, cover art → Music.
`coverage`	Document	Game recaps: sport, matchup, final score, venue → Sports.
`episode`	Document	Optional details for a BTV video (show name, season/episode, featured), matched by YouTube ID → BTV, Music Videos, search.
`radioShow`	Document	A program with weekly time slots — the single source of the schedule, Now On Air, show cards and the live-DJ line.
`person`	Document	One human: author, reviewer, reporter, host or director → Team page and bylines.
`gbm`	Document	General Body Meeting → Join page countdown and list, homepage hero.
`timeSlot`	Object	Day + 24-hour start/end inside a radio show.
`richBody`	Object	Portable Text body: headings, lists, links, inline images, pull quotes.

The legacy `dept` type listed in v1.1 has been removed from the schema; departments are constants in code (§5.8). The Studio's own README still mentions a "Department" row in its content table — a documentation leftover, not a live type.

4.3 References: one person, many roles

Authors and hosts are references to `person` documents, not typed names. Fix a bio or photo once and every byline, show card and director profile updates. Author references are **weak**: if a person is ever deleted, their articles still publish and the site shows the story without a byline instead of failing the whole list.

BEARCAST-STUDIO/SCHEMAS/DOCUMENTS/ARTICLE.TS · LINES 99-109 ARTICLE.TS — A WEAK, REQUIRED AUTHOR REFERENCE

```
99  name: 'author',
100 title: 'Author',
101 type: 'reference',
102 to: [{type: 'person'}],
103 group: 'meta',
104 // weak: true means the article can still be queried/published even
105 // if the referenced person doc is later deleted. Prevents orphan-ref
106 // errors taking down the whole listing. Frontend handles missing
107 // author gracefully.
108 weak: true,
109 validation: (r) => r.required(),
```

4.4 Validation at the source

Every free-text field has a length cap, every URL has a scheme allow-list, colours must be hex, YouTube IDs must be exactly eleven safe characters, and enumerations (desk, sport, verdict, genre, show type) are fixed lists the front end knows how to style. The radio schedule shows how far validation goes: times are 24-hour strings checked by regex, zero-length shows are rejected, overnight shows (22:00 → 02:00) are explicitly allowed, and a show cannot list the same day and start time twice.

BEARCAST-STUDIO/SCHEMAS/OBJECTS/TIMESLOT.TS · LINES 54-77 TIMESLOT.TS — REGEX PLUS A CROSS-FIELD RULE

```
54  name: 'endTime',
55  title: 'End Time',
56  description:
57    '24-hour format. Overnight shows that wrap past midnight are allowed (e.g. 22:00 → 02:00).',
58  type: 'string',
59  validation: (r) =>
60    r
61      .required()
62      .regex(/^([01]\d|2[0-3]):[0-5]\d$/, {
63        name: 'HH:MM (24-hour)',
64        invert: false,
65      })
66      // BUGFIX: reject zero-length shows (start == end), which would
67      // make the "now on air" widget skip the show or display oddly.
68      // Wrap-around (start > end, e.g. 22:00 → 02:00) is intentionally
69      // allowed for overnight shows.
70      .custom((endTime, context: any) => {
71        const startTime = context?.parent?.startTime
72        if (!startTime || !endTime) return true // other validators handle this
73        if (startTime === endTime) {
74          return 'End time must differ from start time (zero-length show).'
75        }
76        return true
77      }),
```

BEARCAST-STUDIO/SCHEMAS/DOCUMENTS/RADIOSHOW.TS · LINES 82-107 RADIOSHOW.TS — DUPLICATE-SLOT DETECTION

```
82  name: 'timeSlots',
83  title: 'Weekly Time Slots',
84  description:
85    'When this show airs. Add one entry per day×time. e.g. Mon 6:00-9:00, Tue 6:00-9:00, etc.',
86  type: 'array',
87  group: 'schedule',
88  of: [{type: 'timeSlot'}],
89  // Detect overlapping slots within the same show (e.g. editor
```

```
// mistakenly adds Mon 6-9 twice). Cross-show overlap detection
// is harder and lives at query time on the frontend.
validation: (r) =>
  r.min(1).custom((slots: any) => {
    if (!Array.isArray(slots)) return true
    // Build a key per slot: day + start. Duplicates indicate overlap.
    const seen = new Map<string, number>()
    for (let i = 0; i < slots.length; i++) {
      const s = slots[i]
      if (!s?.day || !s?.startTime) continue
      const key = `${s.day}:${s.startTime}`
      if (seen.has(key)) {
        return `Duplicate time slot: ${s.day} ${s.startTime} appears more than once.`
      }
      seen.set(key, i)
    }
    return true
  }),
```

Some rules are warnings rather than errors. GBMs normally fall on Thursdays, so another weekday shows a warning the editor can override for a one-off meeting. The date is parsed as a *local* calendar day on purpose: `new Date("2026-10-15")` means midnight UTC, which is still October 14 in Cincinnati.

BEARCAST-STUDIO/SCHEMAS/DOCUMENTS/GBM.TS · LINES 18-28 — GBM.TS — A WARNING, NOT AN ERROR

```
validation: (r) => [
  r.required(),
  // A warning, not an error — a one-off meeting on another day can still be published.
  r.custom((value?: string) => {
    const m = typeof value === 'string' && value.match(/^(\d{4})-(\d{2})-(\d{2})$/)
    if (!m) return true
    const day = new Date(+m[1], +m[2] - 1, +m[3]).getDay()
    const name = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday'][day]
    return day === 4 || `This is a ${name} — GBMs are usually on Thursdays. Double-check the date.`
  }).warning(),
],
```

Slugs are generated from the headline with a strict slugifier (lowercase ASCII, accents stripped, dashes only), giving stable, shareable URLs such as `/journalism?article=the-story`:

BEARCAST-STUDIO/SCHEMAS/DOCUMENTS/ARTICLE.TS · LINES 52-70 — ARTICLE.TS — URL SLUG

```
name: 'slug',
title: 'URL Slug',
type: 'slug',
group: 'meta',
options: {
  source: 'headline',
  maxLength: 96,
  // Reject anything that isn't lowercase/digit/dash; auto-replace
  // spaces with dashes; collapse multiple dashes.
  slugify: (input: string) =>
    input
      .toLowerCase()
      .normalize('NFKD')
      .replace(/[\u0300-\u036f]/g, '') // strip accents
      .replace(/[^a-z0-9]+/g, '-')
      .replace(/^-+|-+$/g, '')
      .slice(0, 96),
},
```

```
70  validation: (r) => r.required(),
```

4.5 The department-shaped desk

Sanity's default navigation lists types alphabetically. `structure.ts` replaces it with the newsroom's shape: Site Settings, then Journalism (All, Featured, Breaking, one list per desk), Music (Reviews, Music Journalism), Sports (Game Coverage, Sports Journalism), BTV, Radio, People (All, Directors, Executive Leadership) and GBM Meetings. Each sub-list is the same underlying type sliced with a GROQ filter, so editors never hunt through one long list.

BEARCAST-STUDIO/STRUCTURE.TS · LINES 67-81 — STRUCTURE.TS — THE FIRST JOURNALISM SUB-LISTS

```
67  S.listItem()
68    .title('Journalism')
69    .icon(DocumentTextIcon)
70    .child(
71      S.list()
72        .title('Journalism')
73        .items([
74          S.listItem()
75            .title('All Articles')
76            .icon(DocumentTextIcon)
77            .child(
78              S.documentTypeList('article')
79                .title('All Articles')
80                .filter('_type == "article"')
81                .defaultOrdering(ORDER_NEWEST_PUBLISHED),
```

BEARCAST-STUDIO/STRUCTURE.TS · LINES 272-281 — STRUCTURE.TS — A LIST THAT MIRRORS WHAT THE FORM CAN EDIT

```
272 .title('Executive Leadership')
273 .child(
274   S.documentTypeList('person')
275     .title('Executive Leadership')
276     // FIX: was filtering on `isExecutive == true` alone,
277     // but the schema only exposes the `isExecutive` field
278     // when `isDirector` is true. Filtering on both makes
279     // the list match what's actually editable in Studio.
280     .filter('_type == "person" && isDirector == true && isExecutive == true')
281     .defaultOrdering([{field: 'name', direction: 'asc'}]),
```

4.6 Singleton enforcement

There must be exactly one `siteSettings` document. Sanity has no built-in singleton, so the Studio config closes both ways a second copy could appear: the per-document menu (duplicate, delete, unpublish are hidden) and the global "+ Create" menu (the template is removed). The desk entry opens the one document by its fixed ID.

BEARCAST-STUDIO/SANITY.CONFIG.TS · LINES 32-41 — SANITY.CONFIG.TS — WHAT A SINGLETON MAY NOT DO

```
32 const SINGLETON_TYPES = new Set(['siteSettings'])
33
34 // Document actions that don't make sense for a singleton — these are
35 // filtered from the action menu so editors can't accidentally orphan or
36 // destroy the singleton document.
37 const SINGLETON_ACTIONS_TO_HIDE = new Set([
38   'duplicate', // would create a second instance
39   'delete', // would remove the singleton
40   'unpublish', // would remove it from the published API → break the live site
41 ])
```

BEARCAST-STUDIO/SANITY.CONFIG.TS · LINES 87-114 — SANITY.CONFIG.TS — BOTH PATHS CLOSED

```
87 document: {
88   actions: (prev, context) => {
89     if (SINGLETON_TYPES.has(context.schemaType)) {
90       return prev.filter((action) => !SINGLETON_ACTIONS_TO_HIDE.has(action.action || ''))
91     }
92     return prev
93   },
94
95   newDocumentOptions: (prev, {creationContext}) => {
   ⋮ … 9 lines omitted …
105     if (creationContext.type === 'global') {
106       return prev.filter(
107         (template) =>
108           !SINGLETON_TYPES.has(template.templateId) &&
109           !SINGLETON_TYPES.has((template as any).schemaType),
110       )
111     }
112     return prev
113   },
114 },
```

4.7 Portable Text — the richBody object

Long-form bodies are stored as Portable Text, Sanity's structured JSON for rich text, rather than an HTML string. The schema allows paragraphs, H2–H4, blockquotes, two list types, five inline decorators, links, inline images (alt text required) and a custom pull quote. Link schemes are restricted to `http`, `https`, `mailto` and `tel` at input time, so a `javascript:` URL can never be saved.

BEARCAST-STUDIO/SCHEMAS/OBJECTS/RICHBODY.TS · LINES 46-74 — RICHBODY.TS — THE LINK ANNOTATION

```
46 annotations: [
47   {
48     name: 'link',
49     type: 'object',
50     title: 'Link',
51     fields: [
52       {
53         name: 'href',
54         type: 'url',
55         title: 'URL',
56         // SECURITY: explicitly restrict URL schemes. Sanity's
57         // default `url` validator allows ftp, irc, and others
58         // and (depending on version) does NOT block javascript:.
59         // Hard-allowlist only the schemes the site actually uses.
60         validation: (r: any) =>
```

```
          r
            .required()
            .uri({
              scheme: ['http', 'https', 'mailto', 'tel'],
              allowRelative: true,
            }),
      },
      {
        name: 'openInNewTab',
        type: 'boolean',
        title: 'Open in new tab',
        initialValue: false,
      },
    ],
```

The site's renderer for this format is in §6.5.

4.8 Studio deployment and upgrades

The Studio is a separate app (Sanity v6.18, React 19) deployed with `npm run deploy` to `bearcast.sanity.studio`; editor-only code never ships to visitors. Dependencies are pinned to exact versions, with `overrides` forcing patched releases of three transitive packages. `deployment.autoUpdates` lets the hosted Studio take patch releases inside the current major version without a redeploy; schema changes still go through review.

BEARCAST-STUDIO/SANITY.CLI.TS · LINES 9-44 — SANITY.CLI.TS

```
export default defineCliConfig({
  api: {
    // Allow env-var override so a future `staging` or `preview` dataset
    // doesn't accidentally hit production. Defaults to the production
    // values for normal local development.
    projectId: process.env.SANITY_STUDIO_PROJECT_ID || '3getuuyu',
    dataset: process.env.SANITY_STUDIO_DATASET || 'production',
  },

… 5 lines omitted …
  studioHost: 'bearcast',

… 15 lines omitted …
  deployment: {
    appId: 'pkofvyqus27yytcrig3kzci7', // Avoids prompting for application id on deploy
    autoUpdates: true,
  },
})
```

BEARCAST-STUDIO/PACKAGE.JSON · LINES 13-20 — PACKAGE.JSON — PINNED VERSIONS AND SECURITY OVERRIDES

```
"dependencies": {
  "@sanity/icons": "5.2.3",
  "@sanity/vision": "6.18.0",
  "react": "19.3.0",
  "react-dom": "19.3.0",
  "sanity": "6.18.0",
  "styled-components": "6.5.3"
},
```

4.9 Content migrations

Bulk content fixes are code, kept in `bearcast-studio/migrations/` so every change is reviewable and reversible. Each change file has a matching `*.ROLLBACK.json` with the previous values. Field edits run through `apply-patches.mjs`, which also patches any unpublished draft so a later "Publish" cannot bring the old values back; structural moves (turn a review into a music-journalism article, delete the review) run through `apply-mutations.mjs`, one transaction per document, so each move happens completely or not at all. Both support `--dry`.

BEARCAST-STUDIO/MIGRATIONS/APPLY-PATCHES.MJS · LINES 20-37 — APPLY-PATCHES.MJS — DRAFT-AWARE, BATCHED TRANSACTIONS

```
const patches = JSON.parse(readFileSync(file, 'utf8'))
const client = getCliClient({apiVersion: '2024-01-01'})
const ids = patches.map((p) => p.id)
const drafts = new Set(await client.fetch('*[_id in $ids]._id', {ids: ids.map((id) => 'drafts.' + id)}))
const existing = new Set(await client.fetch('*[_id in $ids]._id', {ids}))
const missing = ids.filter((id) => !existing.has(id))
if (missing.length) console.warn(`Skipping ${missing.length} document(s) that no longer exist:`, missing.slice(0, 5))

let done = 0
for (let i = 0; i < patches.length; i += 50) {
  const tx = client.transaction()
  for (const p of patches.slice(i, i + 50)) {
    if (!existing.has(p.id)) continue
    tx.patch(p.id, (q) => q.set(p.set))
    if (drafts.has('drafts.' + p.id)) tx.patch('drafts.' + p.id, (q) => q.set(p.set))
    done++
  }
  if (!dry) await tx.commit({visibility: 'async'})
```

BEARCAST-STUDIO/MIGRATIONS/2026-10-09-MOVE-SPORTS-JOURNALISM.JSON · LINES 1-24 — A MOVE: COVERAGE → SPORTS-JOURNALISM ARTICLE, SAME SLUG

```
[
{
"label": "Preseason NBA Power Rankings",
"mutations": [
{
"create": {
"_id": "sj-wp-coverage-17803",
"_type": "article",
"desk": "SportsJournalism",
"category": "Analysis",
"headline": "Preseason NBA Power Rankings",
"slug": {
"_type": "slug",
"current": "preseason-nba-power-rankings"
},
"author": {
"_ref": "person-peytonr",
"_type": "reference"
},
"publishedAt": "2025-10-21T19:05:15.000Z",
"tags": [
"Analysis",
"Basketball"
],
```

MIGRATION (OCT 2026)	EFFECT
`2026-10-08-fix-genres`	Consistent review genres and coverage sports; journalism desks, categories and tags corrected; 11 music and 9 sports articles moved to their pages.
`2026-10-08-fix-artists-albums`	Real artist and album for 183 reviews that read “Bearcast Music Contributor” or repeated the headline.
`2026-10-09-move-music-journalism`	110 non-review pieces (interviews, galleries, playlists, recaps, Queen City Sessions…) became Music Journalism articles; 94 real reviews stay.
`2026-10-09-move-sports-journalism`	66 non-game pieces (picks, rankings, columns…) became Sports Journalism; one recap moved the other way; 184 game pieces stay coverage.

Because moved documents keep their slug, old links keep working: when a page cannot find `?article=` locally it asks Sanity where that slug lives now and forwards the reader (§6.6).

§ 05 — FRONTEND

Frontend Code & Page Architecture

> *Nine hand-written HTML pages share one stylesheet and one script. Each page announces itself with* `<body data-page="…">`*; every style and every page module keys off that attribute.*

5.1 Pages

FILE	DATA-PAGE	WHAT IT DOES
`index.html`	home	25th-anniversary hero with radio player, live waveform and sticky mini player; next GBM; rotating “latest from Bearcast”; Now On Air and today’s schedule; departments.
`radio.html`	radio	Full player with spectrum, VU meter and visualizer styles; DJ-set detection and set timer; sleep timer; listener count; schedule; Our Shows; recent tracks with Spotify / Apple Music links; song requests; Your Bearcast.
`journalism.html`	journalism	*The Bearcast* — a newspaper front with nameplate, edition line, desk pills, grid/list, sort, reader.
`music.html`	music	Reviews (stars, verdicts) and music journalism, plus BTV music videos in a pop-up.
`sports.html`	sports	Game coverage with a scoreboard header and sports journalism; sport filter; “Follow” on X.
`btv.html`	btv	Every video on the YouTube channel, view counts, show filters, featured hero, privacy-enhanced player.
`team.html`	team	Directors in seniority order with full-profile pop-ups.
`join.html`	join	Recruiting, GBM countdown and schedule, interest form.
`404.html`	404	“Signal lost” page with fuzzy route suggestions — and the offline fallback.

5.2 One file, page-scoped

Each page module is an immediately-invoked function that exits on the first line unless it is on its own page. This is what lets one `scripts.js` serve nine pages safely, and what lets the build cut it apart (§13.2):

SCRIPTS.JS · LINES 3929-3931 — THE GUARD AT THE TOP OF EVERY PAGE MODULE

```
3929  //#page radio — build.sh puts this module only in the radio page's bundle
3930  (function(){
3931    if (!document.body || document.body.dataset.page !== 'radio') return;
```

Styles follow the same rule: every page-specific selector begins with `body[data-page="…"]`, so a rule written for one page can never match another. Theme colours are CSS variables set per theme:

STYLES.CSS · LINES 75-105 — STYLES.CSS — PAGE-SCOPED THEME TOKENS

```
/* ─── THEME VARIABLES ──────────────────────────────────────── */
body[data-page="home"]{
  --red: #D0021B;
  --red-bright: #FF1F3D;
  --red-muted: #8B0010;
  --gold: #C9A84C;
  --gold-light: #F0C97A;
  --white: #FFFFFF;
  --off-white: #F5F4F0;
  --near-black: #0A0A0B;
  --easing: cubic-bezier(0.25, 0.46, 0.45, 0.94);
  --easing-spring: cubic-bezier(0.34, 1.56, 0.64, 1);
}

html[data-theme="dark"] body[data-page="home"]{
  --bg: #0A0A0B;
  --bg-2: #111114;
  --bg-3: #1A1A1E;
  --bg-glass: rgba(16,16,20,0.8);
  --fg: #F0EDE8;
  --fg-2: #A8A4A0;
  --fg-3: #5A5754;
  --border: rgba(255,255,255,0.08);
  --border-bright: rgba(255,255,255,0.15);
  --shadow: rgba(0,0,0,0.6);
  --glow-red: rgba(208,2,27,0.3);
  --card-bg: rgba(22,22,28,0.9);
  --nav-bg: rgba(10,10,11,0.92);
}

html[data-theme="light"] body[data-page="home"]{
```

5.3 Actions without inline JavaScript

The Content-Security-Policy forbids inline event handlers (`onclick=""`), so buttons name an action instead: `data-act="functionName"` with an optional `data-arg` . One delegated listener at the top of `scripts.js` looks the name up in `window.bcAct` , which each page module fills with its own functions. Forms use `data-submit` the same way.

SCRIPTS.JS · LINES 29-38 — SCRIPTS.JS — THE ACTION DISPATCHER

```
// Page actions for buttons marked data-act="name" (with optional data-arg)
// and forms marked data-submit="name". No inline onclick="" anywhere, so the
// Content-Security-Policy can forbid inline script. Each page module adds its
// handlers with Object.assign(window.bcAct, { … }).
window.bcAct = window.bcAct || {};
document.addEventListener('click', e => {
  const el = e.target.closest && e.target.closest('[data-act]');
  const fn = el && window.bcAct[el.dataset.act];
  if (typeof fn === 'function') fn(el.hasAttribute('data-arg') ? el.dataset.arg : e);
});
```

BTV.HTML · LINE 274 — BTV.HTML — A BUTTON THAT NAMES ITS ACTION

```
<button class="vbtn active" id="gridVBtn" data-act="setView" data-arg="grid" title="Grid">
```

SCRIPTS.JS · LINE 8454 — MUSIC MODULE — REGISTERING ITS ACTIONS

```
8454  Object.assign(window.bcAct, { closeModal, copyLink, handleOverlayClick, loadMore, nextArticle, prevArticle,
      openFeatured, setView, shareArt, switchMode });
```

A pull-request check rejects any HTML with an inline handler (§14.1), so this pattern cannot quietly regress.

5.4 The shared navigation

Every page carries the same `<nav class="bc-nav">`: a desktop menubar, a theme button, a search button and a burger that opens a slide-in panel styled as a channel changer (CH·01 Radio … CH·05 Music). The current page is marked with both `data-active="true"` (styling) and `aria-current="page"` (screen readers). Each department page swaps in its own wordmark (Bearcast Radio, Bearcast TV, …).

RADIO.HTML · LINES 108-116 — RADIO.HTML — DESKTOP LINKS

```
108  <ul class="bcn-links" role="menubar">
109    <li role="none"><a class="bcn-link" data-dept="radio" data-active="true" aria-current="page"
     href="radio.html" role="menuitem">Radio</a></li>
110    <li role="none"><a class="bcn-link" data-dept="sports" href="sports.html" role="menuitem">Sports</a></li>
111    <li role="none"><a class="bcn-link" data-dept="btv" href="btv.html" role="menuitem">BTV</a></li>
112    <li role="none"><a class="bcn-link" data-dept="journalism" href="journalism.html"
     role="menuitem">Journalism</a></li>
113    <li role="none"><a class="bcn-link" data-dept="music" href="music.html" role="menuitem">Music</a></li>
114    <li role="none"><a class="bcn-link" data-dept="team" href="team.html" role="menuitem">Team</a></li>
115    <li role="none"><a class="bcn-link" data-dept="join" href="join.html" role="menuitem">Join Us</a></li>
116  </ul>
```

The nav behaviour (open/close, scrim, Escape, closing on resize past 920 px, theme buttons) lives once at the bottom of `scripts.js`, outside every page gate, so it runs on all pages.

5.5 Theme: Auto, Light, Dark

The theme button cycles Auto → Light → Dark. Auto follows the operating system and keeps following it live. Two attributes on `<html>` carry the state: `data-theme-mode` (the choice) and `data-theme` (what is painted). To avoid a flash of the wrong theme, a tiny bootstrap runs synchronously in every page's `<head>` before first paint. It is one of only two inline scripts on the site, and the CSP allows it by its exact SHA-256 hash.

INDEX.HTML · LINES 30-49 — INDEX.HTML — PRE-PAINT THEME BOOTSTRAP (IDENTICAL ON EVERY PAGE)

```
30  <script>
31  // Bearcast theme bootstrap — synchronous, pre-paint. Do NOT defer.
32  // Modes: 'auto' (follow the browser/OS setting), 'light', 'dark'.
33  // Default is auto; data-theme-mode is the choice, data-theme is what paints.
34  (function(){
35    var mode = 'auto';
36    try {
37      var saved = localStorage.getItem('bc-theme');
38      if (saved === 'auto' || saved === 'light' || saved === 'dark') mode = saved;
39    } catch (e) { /* private mode etc — fall back to auto */ }
40    var theme = mode;
41    if (mode === 'auto') {
42      theme = (window.matchMedia && window.matchMedia('(prefers-color-scheme: light)').matches)
43        ? 'light'
44        : (document.documentElement.getAttribute('data-theme') || 'dark');
45    }
```

```
  document.documentElement.setAttribute('data-theme-mode', mode);
  document.documentElement.setAttribute('data-theme', theme);
})();
</script>
```

SCRIPTS.JS · LINES 2195-2210 — SCRIPTS.JS — APPLYING AND ANNOUNCING A THEME

```
function setTheme(mode, persist){
  if (ORDER.indexOf(mode) === -1) mode = 'auto';
  var theme = mode === 'auto' ? systemTheme() : mode;
  var root = document.documentElement;
  root.setAttribute('data-theme-mode', mode);
  root.setAttribute('data-theme', theme);
  var meta = document.querySelector('meta[name="theme-color"]');
  if (meta) meta.setAttribute('content', THEME_COLOR[theme]);
  if (persist !== false) {
    try { localStorage.setItem('bc-theme', mode); } catch(e){}
  }
  syncLabels(mode, theme);
  document.dispatchEvent(new CustomEvent('bc-theme-change', {
    detail: { mode: mode, theme: theme }
  }));
}
```

Components that draw with canvas (the visualizers, share cards) listen for the `bc-theme-change` event and recolour without a reload.

5.6 Typography

All fonts are self-hosted in `/fonts` (Latin and Latin-Extended subsets, `font-display: swap`), so no request ever goes to Google. The base system is Playfair Display for display type, DM Sans for text and DM Mono for labels; three departments add a voice of their own: Libre Baskerville on Journalism (the newspaper), Cormorant Garamond on Music and Sports, and Bebas Neue on BTV (a broadcast title face). Each page preloads only the two fonts its first screen needs.

INDEX.HTML · LINES 62-63 — INDEX.HTML — PRELOADING TWO FONTS

```
<link rel="preload" href="/fonts/dm-sans-latin-f71cbe.woff2" as="font" type="font/woff2" crossorigin>
<link rel="preload" href="/fonts/playfair-display-latin-fdd6c3.woff2" as="font" type="font/woff2"
crossorigin>
```

5.7 Accessibility building blocks

HELPER	WHAT IT GUARANTEES
`bcDialog`	One pop-up for show details, director profiles and videos, built on native `<dialog>`: Escape, focus trapping and backdrop for free; focus returns to the opener; closing empties the body (stopping any video).
`bcFocusTrap` / `bcFocusRelease`	Tab stays inside the article reader, mobile panel and ⌘K palette; prior focus is restored on close.
`bcActivatable`	Clickable cards become focusable buttons; Enter/Space activate them via one delegated listener.
`bcToast`	Non-blocking notices in an `aria-live="polite"` region instead of `alert()`.

HELPER	WHAT IT GUARANTEES
Skip link	`<a href="#main-content" class="bc-skip-link">` on every page.
Reduced motion	Custom cursor, party mode, waveform animation and smooth scrolling all stand down for `prefers-reduced-motion`.
Small screens	`bcInitMobileFilter` folds long filter-pill rows into a “Filter” dropdown; pages work down to 320 px.

SCRIPTS.JS · LINES 1909-1922 — SCRIPTS.JS — KEYBOARD-ACTIVATABLE CARDS

```
1909 window.bcActivatable = function(el){
1910   if (!el) return el;
1911   el.tabIndex = 0;
1912   el.setAttribute('role', 'button');
1913   el.setAttribute('data-bc-activate', '');
1914   return el;
1915 };
1916 document.addEventListener('keydown', function(e){
1917   if (e.key !== 'Enter' && e.key !== ' ') return;
1918   var t = e.target;
1919   if (!t || !t.hasAttribute || !t.hasAttribute('data-bc-activate')) return;
1920   e.preventDefault(); // stop Space from scrolling the page
1921   t.click();
1922 });
```

5.8 The custom desktop cursor

On mouse-driven devices the operating-system pointer is replaced by an SVG arrow that grows over interactive elements, springs on click and steps aside over text fields. It installs only when the user has not asked for reduced motion and the pointer is fine (a mouse, not a finger). Movement is batched into one `requestAnimationFrame` write per frame using `translate3d`, so it never triggers layout.

SCRIPTS.JS · LINES 2839-2857 — SCRIPTS.JS — RAF-BATCHED MOVEMENT; NATIVE CURSOR HIDDEN ONLY AFTER THE FIRST MOVE

```
2839 var cx = -100, cy = -100, scheduled = false, hasMoved = false;
2840 document.addEventListener('mousemove', function (e) {
2841   cx = e.clientX; cy = e.clientY;
2842   if (!scheduled) {
2843     scheduled = true;
2844     requestAnimationFrame(function () {
2845       scheduled = false;
2846       rehost();
2847       el.style.transform = 'translate3d(' + cx + 'px,' + cy + 'px,0)';
2848       if (!hasMoved) {
2849         // First movement: the pointer position is now known, so swap
2850         // the native cursor for the custom one — no corner flash.
2851         hasMoved = true;
2852         document.documentElement.classList.add('bc-cursor-active');
2853         el.style.opacity = '1';
2854       }
2855     });
2856   }
2857 }, { passive: true });
```

Two refinements since v1.1 fix real bugs. A modal `<dialog>` renders in the browser's top layer above any z-index, so while one is open the cursor element is moved inside it (`rehost()`). And Safari does not reliably fire `mouseleave` on the document, so "pointer left the page" is detected from `mouseout` / `pointerout` events whose `relatedTarget` is null, plus window blur.

STYLES.CSS · LINES 1278-1286 — STYLES.CSS — HIDE THE OS CURSOR, BUT KEEP THE TEXT CARET

```
1278 html.bc-cursor-active *{
1279   cursor: none !important;
1280 }
1281
1282 /* Restore I-beam in text fields — users need native precision */
1283 html.bc-cursor-active input, html.bc-cursor-active textarea, html.bc-cursor-active [contenteditable]{
1284   cursor: text !important;
1285 }
1286
```

5.9 Hardcoded departments

The departments are not CMS content. Each corresponds to an HTML file, a nav link, a colour and a font, so letting editors rename or add one would create links to pages that do not exist. The Join form's list is the clearest example — six interest areas (the five departments plus Operations), defined in code with their colours:

SCRIPTS.JS · LINES 10230-10237 — JOIN MODULE — INTEREST AREAS

```
10230 const DEPTS = [
10231   {name:'Radio',      color:'#FF1F3D', emoji:'📻'},
10232   {name:'Sports',     color:'#FF6B35', emoji:'🏟️'},
10233   {name:'Music',      color:'#845EF7', emoji:'🎵'},
10234   {name:'Journalism', color:'#339AF0', emoji:'📰'},
10235   {name:'BTV',        color:'#51CF66', emoji:'📺'},
10236   {name:'Operations', color:'#C9A84C', emoji:'⚙️'},
10237 ];
```

> NOT A SIXTH DEPARTMENT
>
> The homepage shows "Six Frequencies": the five departments plus Music Journalism. Music Journalism is a *desk* (a value of `article.desk`) that appears on the Music page. Departments are pages; desks slice content within them.

5.10 Small things that add up

- **Footer year.** Every footer's `<span data-bc-year>2026</span>` is rewritten to the current year; the typed year remains if JavaScript is off.
- **Deep links.** `bcSetDeepLink` / `bcGetDeepLink` keep `?article=`, `?ep=` and search parameters in the address bar via `history.replaceState`, so anything open on screen can be copied and shared.
- **Reader keys.** With a story open, ← / → move between stories and Escape closes, on every page with a reader.
- **Konami code.** ↑↑↓↓←→←→BA gives ten seconds of "party mode" — skipped for reduced-motion users.
- **Offline notices.** Toasts say when the connection drops ("cached pages still work, live radio won't") and when it returns.

§ 06 — DATA FLOW

Loading Content from Sanity

> *All content reaches the page through one function, one query language and one status contract. Pages show a designed loading, empty or error state — never a blank one.*

6.1 sanityFetch — the single entry point

Every query goes to Sanity's API CDN (`apicdn.sanity.io`), where results are edge-cached for about a minute — faster than the live API, and a minute of freshness is right for published journalism. `perspective=published` guarantees drafts are never returned, even with a valid draft ID. Parameters are passed as GROQ `$parameters` (JSON-encoded, escaped by Sanity), never pasted into the query string. An eight-second timeout stops a hung request from blocking the page, and any failure returns `null` so callers can fall back.

SCRIPTS.JS · LINES 755-792 — SCRIPTS.JS — SANITYFETCH

```
window.sanityFetch = async function(query, params){
  if (!window.BEARCAST_SANITY_ENABLED) {
    console.warn('[Bearcast] sanityFetch called but disabled. Returning null.');
    return null;
  }
  const pid = window.SANITY_PROJECT_ID;
  const ds  = window.SANITY_DATASET;
  const ver = window.SANITY_API_VERSION || '2024-01-01';
  if (!pid || pid === 'YOUR_PROJECT_ID') {
    console.warn('[Bearcast] SANITY_PROJECT_ID not set. Returning null.');
    return null;
  }
  // apicdn is the cached read endpoint — fast, no auth, public docs only
  const base = `https://${pid}.apicdn.sanity.io/v${ver}/data/query/${ds}`;
  const qs = new URLSearchParams();
  qs.set('query', query);
  qs.set('perspective', 'published');
  if (params && typeof params === 'object') {
    for (const k in params) {
      qs.set('$' + k, JSON.stringify(params[k]));
    }
  }
  const url = base + '?' + qs.toString();
  // 8-second timeout so a hung request doesn't block the page forever
  const ctrl = new AbortController();
  const t = setTimeout(() => ctrl.abort(), 8000);
  try {
    const res = await fetch(url, {signal: ctrl.signal});
    if (!res.ok) throw new Error('Sanity HTTP ' + res.status);
    const data = await res.json();
    return data.result;
  } catch (e) {
    console.error('[Bearcast] sanityFetch error:', e?.message || e);
    return null;
  } finally {
    clearTimeout(t);
  }
};
```

6.2 GROQ: ask for exactly what the page renders

Each page sends one projection-shaped query and receives JSON in the shape its renderer expects. References are joined in the same request (`author->name`). The Journalism page, for example:

SCRIPTS.JS · LINES 9631-9652 — JOURNALISM MODULE — THE PAGE QUERY

```
9631    *[_type == "article" && !(desk in ["MusicJournalism","SportsJournalism"])]
9632    | order(publishedAt desc) {
9633      "id": _id,
9634      "slug": slug.current,
9635      featured,
9636      breaking,
9637      desk,
9638      "category": category,
9639      "emoji": coalesce(emoji, "📰"),
9640      "headline": headline,
9641      "deck": deck,
9642      excerpt,
9643      "author": author->name,
9644      "authorRole": coalesce(author->role, "Staff Reporter"),
9645      "date": publishedAt,
9646      "words": ${window.BC_GROQ_WORDS},
9647      tags,
9648      coverImage,
9649      "imageAlt": coalesce(coverImage.alt, headline),
9650      "imageCaption": coverImage.caption
9651    }
9652  `;
```

Notice `"words"` . Lists used to download every full article body (about 1 MB per page) just to compute reading time. Now Sanity counts the words server-side with a GROQ expression, and the body is fetched only when a reader opens a story:

SCRIPTS.JS · LINES 1357-1372 — SCRIPTS.JS — WORD COUNT IN GROQ; BODIES LOADED ON DEMAND

```
1357  // GROQ: an article's word count, computed by Sanity (line breaks, tabs and
1358  // non-breaking spaces count as spaces) — matches counting the full text.
1359  window.BC_GROQ_WORDS =
      String.raw`count(string::split(array::join(string::split(array::join(string::split(array::join(string::split(p
      "\n"), " "), "\t"), " "), "\u00a0"), " "), " ")[@ != ""])`;
1360  window.bcArticleText = (function(){
1361    const bodies = new Map(), texts = new Map();
1362    const once = (cache, key, load) => {
1363      if (!cache.has(key)) cache.set(key, load().catch(e => { cache.delete(key); throw e; }));
1364      return cache.get(key);
1365    };
1366    return {
1367      body: id => once(bodies, id, () => window.sanityFetch('*[_id == $id][0].body', { id }).then(b =>
      (Array.isArray(b) ? b : [])),
1368      // filter is a fixed GROQ filter from this file (never user input)
1369      texts: filter => once(texts, filter, () => window.sanityFetch(`*[${filter}]{ "id": _id, "text":
      pt::text(body) }`)
1370        .then(rows => new Map((rows || []).map(r => [r.id, r.text || '']))),
1371    };
1372  })();
```

SCRIPTS.JS · LINES 1373-1384 — SCRIPTS.JS — FILLING THE READER, WITH A RACE GUARD

```
1373 window.bcReaderBody = function(el, item){
1374   if (!el || !item) return;
1375   el.dataset.for = String(item.id);
1376   if (item.content != null) { el.innerHTML = window.bcSanitizeHtml(item.content); return; }
1377   el.innerHTML = '<p class="reader-loading">Loading the story…</p>';
1378   window.bcArticleText.body(item.id).then(body => {
1379     item.content = window.renderPortableText(body);
1380     if (el.dataset.for === String(item.id)) el.innerHTML = window.bcSanitizeHtml(item.content);
1381   }).catch(() => {
1382     if (el.dataset.for === String(item.id)) el.innerHTML = '<p class="reader-loading">Couldn't load this
     story — check your connection and try again.</p>';
1383   });
1384 };
```

The `el.dataset.for` check prevents a slow response for story A from overwriting story B if the reader moved on in the meantime. GROQ was chosen over Sanity's GraphQL API because it needs no schema deployment step and because the Studio's desk filters (§4.6) are already GROQ — one query language for editors' lists and visitors' pages.

6.3 The status contract

`bcLoadWithStatus(query)` returns `{ status, items }` where status is `ok`, `empty` (Sanity answered with nothing) or `error` (unreachable). Pages copy the status onto `<body data-bc-status>`, and `bcUpdateEmptyState` rewrites the page's empty-state block to match: "Loading articles…", "No articles yet", "Couldn't load articles", or — when a filter or search hides everything — "No articles match".

SCRIPTS.JS · LINES 1933-1945 — SCRIPTS.JS — LOAD WITH AN EXPLICIT STATUS

```
1933 window.bcLoadWithStatus = async function(query){
1934   try {
1935     const result = await window.sanityFetch(query);
1936     if (result == null) return { status: 'error', items: [] };
1937     if (!Array.isArray(result) || result.length === 0) {
1938       return { status: 'empty', items: [] };
1939     }
1940     return { status: 'ok', items: result };
1941   } catch (e) {
1942     console.error('[Bearcast] bcLoadWithStatus error:', e?.message || e);
1943     return { status: 'error', items: [] };
1944   }
1945 };
```

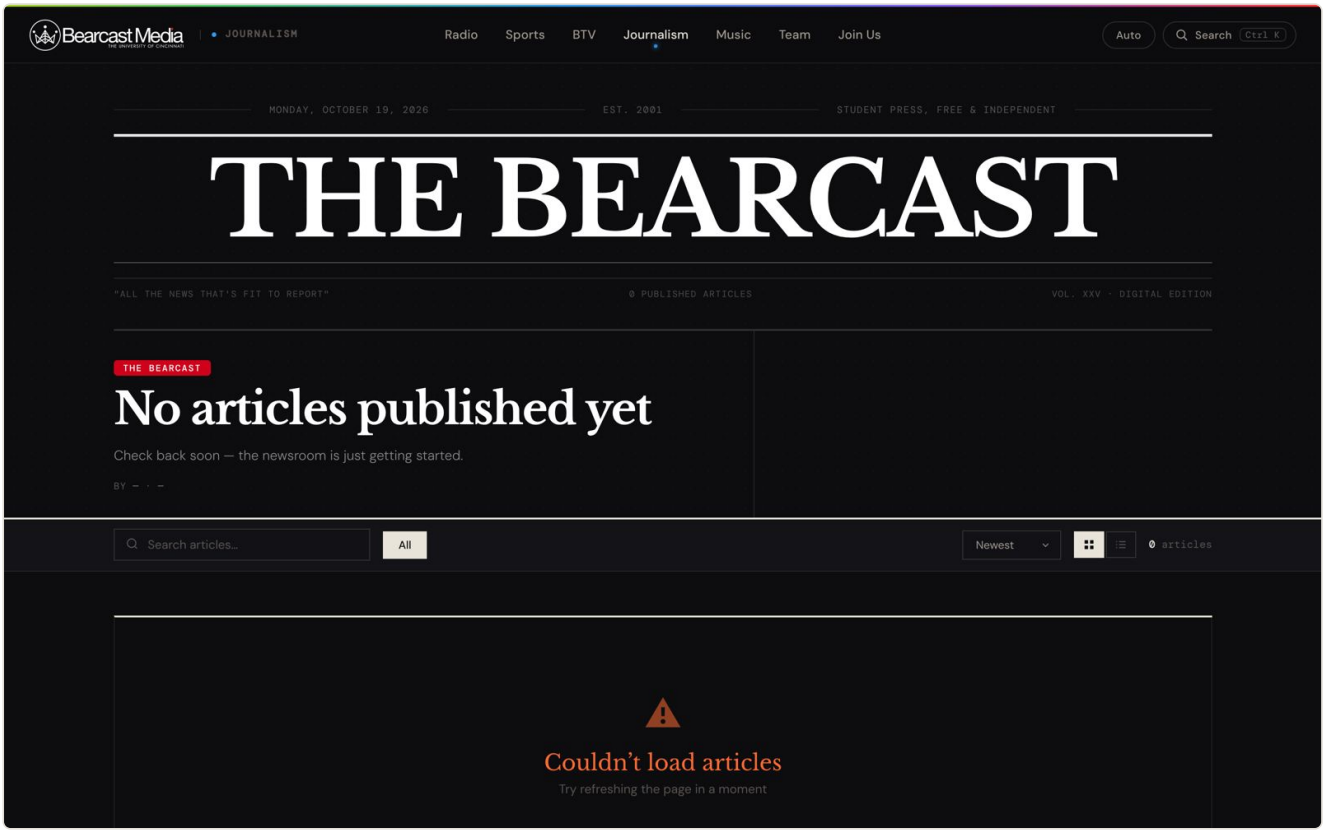


Figure 6.1 — The status contract in action. The Journalism page rendered from the current source with Sanity unreachable: the nameplate, edition line, desk bar and search render from HTML, and the grid shows "Couldn't load articles" instead of an empty frame.

An older helper, `bcLoadCollection(query, fallback)`, returns a fallback array when Sanity is empty or down. The fallbacks that remain (`DIRECTORS_FALLBACK` , `RADIO_SCHEDULE_FALLBACK` , the GBM dates) are now empty or computed rather than fake content: an outage shows an honest empty state, never invented articles.

6.4 Images: the editor's crop and hotspot, respected

Sanity stores an image as an asset reference such as `image-<sha1>-1600x900-jpg` plus optional `crop` and `hotspot` values set in the Studio. `urlForImage` builds a CDN URL with the requested size and turns the crop into a `rect=` parameter and the hotspot into a focal point, so a square card or a 1200×630 link preview keeps the subject in frame.

SCRIPTS.JS · LINES 1592-1627 — SCRIPTS.JS — URLFORIMAGE

```
1592  const ref = img.asset._ref;
1593  const m = ref.match(/^image-([a-zA-Z0-9]+)-(\d+x\d+)-(\w+)$/);
1594  if (!m) return null;
1595  const [, id, dims, ext] = m;
1596  const pid = window.SANITY_PROJECT_ID;
1597  const ds = window.SANITY_DATASET;
1598  let url = `https://cdn.sanity.io/images/${pid}/${ds}/${id}-${dims}.${ext}`;
1599  // Allow size constraints: { w: 800, h: 600, fit: 'crop' }
1600  if (opts && typeof opts === 'object') {
1601    const p = new URLSearchParams();
1602    // Respect the crop and focal point editors set in the Studio
1603    // (the "hotspot" circle) so cropped thumbnails keep the subject in frame.
1604    const wh = dims.split('x').map(Number);
```

```
  const c = img.crop;
  if (c && [c.left, c.top, c.right, c.bottom].every(v => typeof v === 'number') && (c.left || c.top ||
c.right || c.bottom)) {
    const rx = Math.round(c.left * wh[0]), ry = Math.round(c.top * wh[1]);
    const rw = Math.round((1 - c.left - c.right) * wh[0]), rh = Math.round((1 - c.top - c.bottom) * wh[1]);
    if (rw > 0 && rh > 0) p.set('rect', [rx, ry, rw, rh].join(','));
  }
  const hs = img.hotspot;
  if (opts.fit === 'crop' && hs && typeof hs.x === 'number' && typeof hs.y === 'number') {
    // fp-x/fp-y are relative to the (already cropped) image.
    const cl = (c && c.left) || 0, cr = (c && c.right) || 0, ct = (c && c.top) || 0, cb = (c && c.bottom)
|| 0;
    const fx = (hs.x - cl) / Math.max(0.01, 1 - cl - cr), fy = (hs.y - ct) / Math.max(0.01, 1 - ct - cb);
    p.set('crop', 'focalpoint');
    p.set('fp-x', Math.min(1, Math.max(0, fx)).toFixed(3));
    p.set('fp-y', Math.min(1, Math.max(0, fy)).toFixed(3));
  }
  if (opts.w) p.set('w', opts.w);
  if (opts.h) p.set('h', opts.h);
  if (opts.fit) p.set('fit', opts.fit);
  if (opts.q) p.set('q', opts.q);
  if (opts.auto) p.set('auto', opts.auto);
  if ([...p.keys()].length) url += '?' + p.toString();
}
return url;
```

6.5 Rendering Portable Text

A small serializer turns Portable Text into HTML. Every piece of text is escaped, link schemes are re-checked, consecutive list items are grouped into `<ul>` / `<ol>`, and custom blocks (inline images, pull quotes) get their own markup. Its output then passes through `bcSanitizeHtml` before reaching the page — two independent layers (see the Cybersecurity White Paper).

SCRIPTS.JS · LINES 1760-1781 — SCRIPTS.JS — MARKS AND LINKS

```
const renderSpan = (span, markDefs) => {
  let html = esc(span.text || '');
  const marks = span.marks || [];
  for (const m of marks) {
    if (m === 'strong') html = `<strong>${html}</strong>`;
    else if (m === 'em') html = `<em>${html}</em>`;
    else if (m === 'underline') html = `<u>${html}</u>`;
    else if (m === 'strike-through') html = `<s>${html}</s>`;
    else if (m === 'code') html = `<code>${html}</code>`;
    else {
      // Look up annotation by key
      const def = (markDefs || []).find((d) => d._key === m);
      if (def && def._type === 'link') {
        // noreferrer in addition to noopener to prevent leaking referrer
        // info to potentially-untrusted CMS-provided link targets.
        const target = def.openInNewTab ? ' target="_blank" rel="noopener noreferrer"' : '';
        html = `<a href="${esc(safeUrl(def.href))}"${target}>${html}</a>`;
      }
    }
  }
  return html;
};
```

SCRIPTS.JS · LINES 1783-1810 — SCRIPTS.JS — BLOCKS

```
const renderBlock = (b) => {
  if (b._type === 'inlineImage') {
    const url = window.urlForImage(b, {w: 1200, auto: 'format'});
    if (!url) return '';
    const alt = esc(b.alt || '');
    const cap = b.caption
      ? `<figcaption>${esc(b.caption)}</figcaption>`
      : '';
    return `<figure><img src="${esc(url)}" alt="${alt}" loading="lazy">${cap}</figure>`;
  }
  if (b._type === 'pullQuote') {
    const cite = b.attribution
      ? `<cite>— ${esc(b.attribution)}</cite>`
      : '';
    return `<blockquote class="pull-quote">${esc(b.quote)}${cite}</blockquote>`;
  }
  if (b._type !== 'block') return '';
  // Regular text block
  const inner = (b.children || [])
    .map((c) => renderSpan(c, b.markDefs))
    .join('');
  const style = b.style || 'normal';
  if (style === 'blockquote') return `<blockquote>${inner}</blockquote>`;
  if (style === 'h2') return `<h2>${inner}</h2>`;
  if (style === 'h3') return `<h3>${inner}</h3>`;
  if (style === 'h4') return `<h4>${inner}</h4>`;
  return `<p>${inner}</p>`;
};
```

6.6 Links that follow a story

Stories move between pages when their desk changes (the October migrations moved 176). If a `?article=` link does not match anything on the current page, `bcFollowStory` asks Sanity where that slug lives now and forwards the reader; only if it truly no longer exists does the page say so.

SCRIPTS.JS · LINES 1528-1542 — SCRIPTS.JS — FORWARDING MOVED STORIES

```
window.bcFollowStory = function(slug, notFoundMsg){
  const here = document.body && document.body.dataset.page;
  const fail = () => window.bcToast(notFoundMsg, { type: 'error' });
  if (!slug || typeof window.sanityFetch !== 'function') return fail();
  window.sanityFetch(
    '*[_type in ["article","review","coverage"] && (slug.current == $k || _id == $k)][0]{_type, desk, "slug": slug.current}',
    { k: String(slug).slice(0, 200) }
  ).then(function(d){
    if (!d) return fail();
    const page = d._type === 'review' || d.desk === 'MusicJournalism' ? 'music'
      : d._type === 'coverage' || d.desk === 'SportsJournalism' ? 'sports' : 'journalism';
    if (page === here) return fail();
    location.replace(page + '.html?article=' + encodeURIComponent(d.slug || slug));
  }).catch(fail);
};
```

6.7 Dates are calendar days

Sanity `date` fields are plain `YYYY-MM-DD` strings. `new Date("2026-11-27")` means midnight UTC — the evening of the 26th in Cincinnati — which once made a GBM show a day early. `bcParseDate` reads plain dates as local days and full timestamps normally.

SCRIPTS.JS · LINES 1551-1555 — SCRIPTS.JS — BCPARSEDATE

```
1551 window.bcParseDate = function(v){
1552   const m = typeof v === 'string' && v.match(/^(\d{4})-(\d{2})-(\d{2})$/);
1553   const d = m ? new Date(+m[1], +m[2] - 1, +m[3]) : new Date(v);
1554   return v && !isNaN(d) ? d : null;
1555 };
```

6.8 Site Settings and social links

Site Settings rarely change, so the document is cached in `localStorage` for 30 days, keyed to the deploy version (read from the script's own `?v=`), so a deploy always starts fresh. Today the site uses its `social` links: `BC_SOCIAL` renders icon rows, handle pills, the BTV "Subscribe" button and the Sports "Follow" button wherever a page has `data-bc-social`. Until any link is filled in the Studio, a built-in list is used; once one is, exactly the Studio's list is used. Every URL must be `https:`; icons are inlined SVG (Simple Icons, CC0), so no social-network script ever loads.

SCRIPTS.JS · LINES 2002-2024 — SCRIPTS.JS — 30-DAY, PER-DEPLOY SETTINGS CACHE

```
2002 window.bcSiteSettings = (function(){
2003   // Site Settings rarely change, so keep them in this browser for 30 days
2004   // instead of asking Sanity on every page view. A deploy (new scripts.js
2005   // ?v= version) starts fresh, so code changes never wait on old settings.
2006   const KEY = 'bc_site_settings_v1';
2007   const TTL = 30 * 24 * 60 * 60 * 1000;
2008   let ver = 'dev';
2009   try { ver = new URL(document.currentScript.src).searchParams.get('v') || 'dev'; } catch(_) {}
2010   try {
2011     const c = JSON.parse(localStorage.getItem(KEY) || 'null');
2012     if (c && c.v === ver && Date.now() - c.at < TTL && c.data) return Promise.resolve(c.data);
2013   } catch(_) {}
2014   return window.sanityFetch(`
2015     *[_type == "siteSettings"][0]{
2016       siteName, tagline, foundedYear, contactEmail,
2017       missionStatement, heroHeadline, heroSubhead,
2018       social, recruitmentOpen, recruitmentCopy, footerCopy
2019     }
2020   `).then(d => {
2021     if (d) { try { localStorage.setItem(KEY, JSON.stringify({ v: ver, at: Date.now(), data: d })); }
     catch(_) {} }
2022     return d;
2023   }).catch(() => null);
2024 })();
```

SCRIPTS.JS · LINES 2130-2139 — BC_SOCIAL — FALLBACK UNTIL THE STUDIO HAS LINKS

```
function renderAll(){ document.querySelectorAll('[data-bc-social]').forEach(render); }
function start(){
  renderAll();
  Promise.resolve(window.bcSiteSettings).then(s => {
    const fromStudio = clean(s && s.social);
    if (Object.keys(fromStudio).length) { links = fromStudio; renderAll(); }
  }).catch(() => {});
}
if (document.readyState === 'loading') document.addEventListener('DOMContentLoaded', start);
else start();
```

§ 07 — SEARCH & READING

Search, the Command Palette & the Reader

> *One search engine serves the ⌘K palette and the Music, Sports and Journalism search boxes. It forgives typos, ranks title hits above body hits, and hands words-inside-articles to Sanity's full-text index.*

7.1 BC_SEARCH — matching and ranking

Queries and fields are normalized (accents removed, lowercase, possessives dropped) and split into words. Every query word must match some word in the item: exactly (1.0), as a prefix while typing (0.8), inside a longer word (0.45), or within a typo budget — one edit for words of 4–7 letters, two for 8+ — using an optimal-string-alignment distance that gives up early once it exceeds the budget. Fields carry weights (title 3, author 2, tags 1.5, body 0.5), and a whole-phrase match earns a bonus, so "chappell roan" in one field beats the two words scattered.

SCRIPTS.JS · LINES 842-855 — BC_SEARCH — HOW ONE QUERY WORD MATCHES ONE ITEM WORD

```
function typoBudget(len){ return len >= 8 ? 2 : len >= 4 ? 1 : 0; }
// How well one query word matches one item word (0…1).
function wordMatch(q, w){
  if (w === q) return 1;
  if (w.startsWith(q)) return q.length >= 2 ? 0.8 : 0.4;
  if (q.length >= 3 && w.indexOf(q) > 0) return 0.45;
  const max = typoBudget(q.length);
  if (max && w[0] === q[0]){
    // Typo in a full word, or in the part typed so far ("volleyb" → "volleyba…").
    if (dist(q, w, max) <= max) return 0.55;
    if (w.length > q.length && dist(q, w.slice(0, q.length), max) <= max) return 0.4;
  }
  return 0;
}
```

SCRIPTS.JS · LINES 869-892 — BC_SEARCH — SCORE

```
function score(item, fieldsFn, q){
  if (!q || !q.length) return 1;
  const f = prepare(item, fieldsFn);
  let total = 0;
  for (const qw of q){
    let best = 0;
    for (const fld of f){
      let m = 0;
      for (const w of fld.ws){
        const s = fld.typos || w.startsWith(qw) || w === qw ? wordMatch(qw, w) : 0;
        if (s > m){ m = s; if (m === 1) break; }
      }
      if (m * fld.w > best) best = m * fld.w;
    }
    if (!best) return 0;        // every word has to match something
    total += best;
  }
  // Whole phrase in one field ("chappell roan") beats scattered words.
  if (q.length > 1){
    const phrase = q.join(' ');
```

```
    for (const fld of f) if (fld.full.indexOf(phrase) >= 0){ total += fld.w * 1.5; break; }
  }
  return total;
}
```

Results are highlighted with `<mark>` around matching words — built from escaped text, so a hostile title cannot inject markup — and a `snippet` shows the sentence around the first match. When nothing matches, `suggest` offers a “did you mean” built from a vocabulary of every word in the index.

7.2 The ⌘K palette

Ctrl K / ⌘K (or the Search button) opens a palette on every page. Its index is the entire archive’s titles and metadata — articles, reviews, game coverage, BTV videos merged with Studio episodes, radio shows and people — fetched in one combined GROQ request (~45 KB) once per browsing session and cached in `sessionStorage`. It also builds “Jump to” shortcuts for every sport, genre and desk, links writers to everything they have published, and includes the visitor’s own liked and heard songs and saved stories, which never leave the device.

SCRIPTS.JS · LINES 10690-10697 — ⌘K — THE ARCHIVE INDEX QUERY

```
const Q = `{
  "articles": *[_type == "article"] | order(publishedAt desc){ "id": _id, "t": headline, "slug": slug.current, desk, category, tags, "by": author->name, "aid": author._ref, "d": publishedAt },
  "reviews":  *[_type == "review"]  | order(publishedAt desc){ "id": _id, "t": title, "slug": slug.current, artist, album, genre, "by": author->name, "aid": author._ref, "d": publishedAt },
  "coverage": *[_type == "coverage"]| order(publishedAt desc){ "id": _id, "t": title, "slug": slug.current, matchup, "score": finalScore, sport, "by": author->name, "aid": author._ref, "d": publishedAt },
  "episodes": *[_type == "episode"] | order(airDate desc){ "id": _id, "t": title, showName, "d": airDate, "yt": youtubeId },
  "shows":    *[_type == "radioShow"]{ "id": _id, name, tagline, genre, hostDisplayName, "hosts": hosts[]->name },
  "people":   *[_type == "person"] | order(name asc){ "id": _id, name, role, dept }
}`;
```

Words *inside* articles are searched on Sanity’s servers instead of downloading every body. After a short debounce, the palette sends a full-text query ranked with GROQ’s `score()` and `boost()`: headline or title hits count four times, album and artist three, summaries 1.5, body text once. The top twelve hits come back with their text so a matching sentence can be shown.

SCRIPTS.JS · LINES 10716-10720 — ⌘K — SERVER-SIDE FULL-TEXT RANKING

```
const FT_Q = `*[_type in ["article","review","coverage","episode"] &&
    [title, headline, album, artist, deck, excerpt, matchup, description, pt::text(body)] match $q]
  | score(boost(title match $q, 4), boost(headline match $q, 4), boost(album match $q, 3), boost(artist match $q, 3),
          boost(deck match $q, 1.5), boost(excerpt match $q, 1.5), boost(description match $q, 1.5), pt::text(body) match $q)
  | order(_score desc)[0...12]{ "id": _id, _score, "body": coalesce(pt::text(body), description, excerpt, "") }`;
```

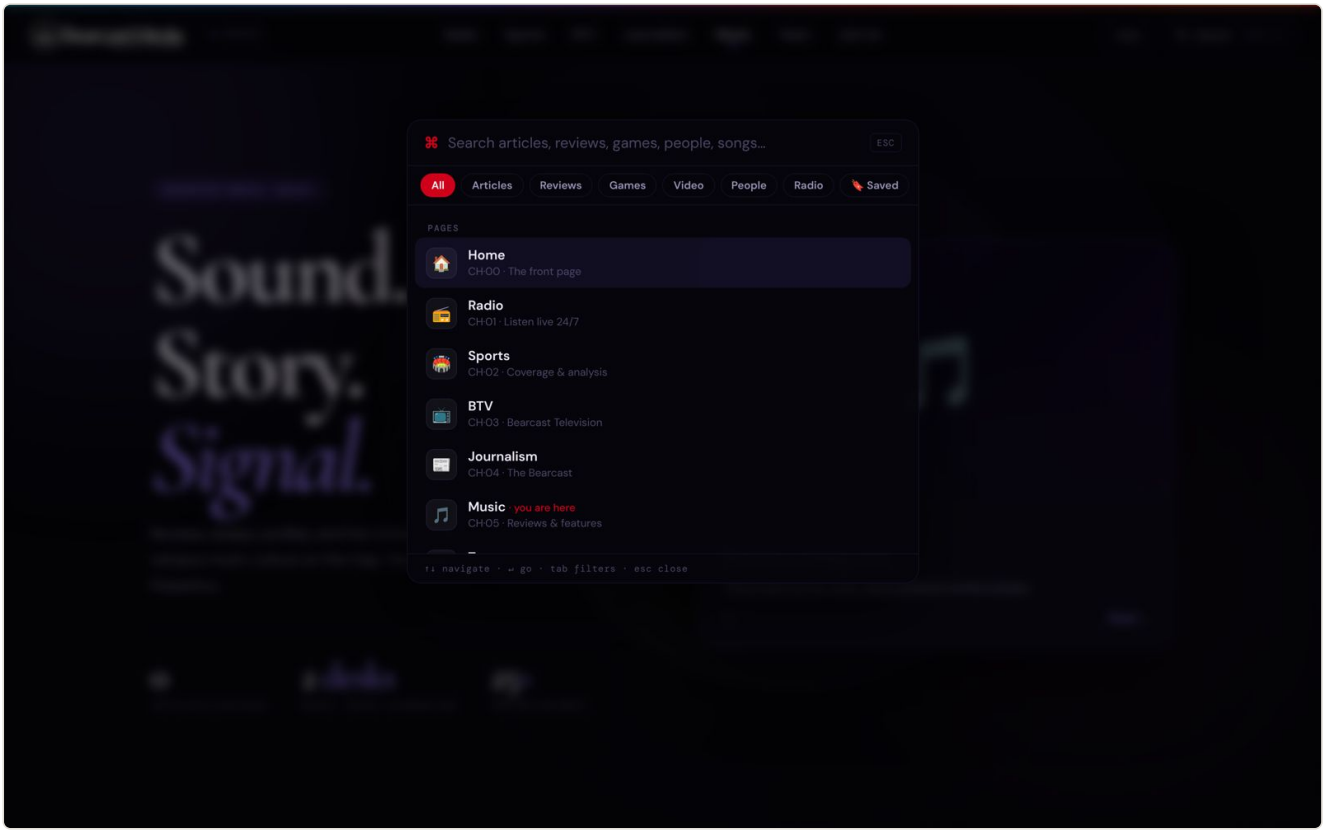


Figure 7.1 — The palette open over the Music page (local render, no CMS connection, so only pages and actions are listed). Filter chips narrow results to Articles, Reviews, Games, Video, People, Radio or Saved.

7.3 Page search boxes

`bcPageSearch` wires a page's search input to the same engine and adds the conveniences visitors expect: the query and active filters stay in the URL (`?q=` , `?sport=` , `?genre=` , `?desk=` , `?author=`) so any search can be shared; / jumps to the box; Escape clears it; focusing an empty box lists the last five searches. The first time someone searches, the page also fetches plain text for its stories (`bcSearchText`) so phrases inside articles match locally too.

SCRIPTS.JS · LINES 994-1004 — BCPAGESEARCH — KEEPING THE URL SHAREABLE

```
994  function syncUrl(){
995    clearTimeout(urlTimer);
996    urlTimer = setTimeout(function(){
997      try {
998        const u = new URL(location.href);
999        const p = Object.assign({ q: input.value.trim() }, opts.params ? opts.params() : {});
1000       Object.keys(p).forEach(k => { if (p[k]) u.searchParams.set(k, p[k]); else
     u.searchParams.delete(k); });
1001       if (u.href !== location.href) history.replaceState(history.state, '', u.pathname + u.search + u.hash);
1002     } catch(_) {}
1003   }, 250);
1004 }
```

7.4 The article reader

Journalism, Music and Sports share one set of reader extras (`bcReader`):

- 🎧 **Listen** reads the story aloud with the device's own voice (Web Speech API — no server, works offline). Text is split into sentence-sized chunks under ~260 characters, because some browsers silently stop long utterances, and the voice picker prefers natural/neural US-English voices.
- 🔖 **Save** keeps a reading list in `localStorage` (`bc_saved_v1` , up to 200), shown as a Saved filter and in ⌘K.
- **Keep reading** suggests three related stories: same writer (+3), same category weighted by how rare it is, shared tags, shared title keywords and closeness in time.

SCRIPTS.JS · LINES 1120-1134 — BCREADER — CHUNKING FOR SPEECH SYNTHESIS

```
1120 function split(text){
1121   const out = [];
1122   (String(text).replace(/\s+/g, ' ').match(/[^.!?…]+[.!?…]+["'”’)\]]*\s*|[^.!?…]+$/g) ||
     []).forEach(sentence => {
1123     let s = sentence.trim();
1124     while (s.length > 260) {
1125       let cut = s.lastIndexOf(', ', 240); if (cut < 80) cut = s.lastIndexOf(' ', 240); if (cut < 80) cut =
     240;
1126       out.push(s.slice(0, cut + 1)); s = s.slice(cut + 1).trim();
1127     }
1128     if (s) {
1129       const last = out[out.length - 1];
1130       if (last && last.length + s.length < 200) out[out.length - 1] = last + ' ' + s; else out.push(s);
1131     }
1132   });
1133   return out;
1134 }
```

SCRIPTS.JS · LINES 1242-1260 — BCREADER — KEEP READING

```
1242 function relatedFor(s, pool){
1243   const n = window.BC_SEARCH.norm;
1244   const kw = keyWords(s.title);
1245   const tags = new Set((s.tags || []).map(t => n(t)));
1246   const t0 = Date.parse(s.date) || 0;
1247   // A category most stories share ("News") says little — weight it by rarity.
1248   const catShare = s.category ? pool.filter(p => p.category && n(p.category) === n(s.category)).length /
     Math.max(1, pool.length) : 1;
1249   const catWeight = 2 * (1 - catShare);
1250   return pool.filter(p => p.id !== s.id && p.title).map(p => {
1251     let sc = 0, why = '';
1252     if (s.author && p.author && n(p.author) === n(s.author)) { sc += 3; why = 'More from ' + s.author; }
1253     if (s.category && p.category && n(p.category) === n(s.category)) { sc += catWeight; if (catShare < 0.3)
     why = why || 'More on ' + s.category; }
1254     (p.tags || []).forEach(t => { if (tags.has(n(t))) sc += 1; });
1255     keyWords(p.title).forEach(w => { if (kw.has(w)) sc += 0.8; });
1256     const t1 = Date.parse(p.date) || 0;
1257     if (t0 && t1) sc += Math.max(0, 1 - Math.abs(t0 - t1) / 3.15e10); // closer in time, a bit more relevant
1258     return { p, sc, why: why || 'You might also like' };
1259   }).filter(x => x.sc >= 1.5).sort((a, b) => b.sc - a.sc).slice(0, 3);
1260 }
```

§ 08 — AUDIO

Radio: Player, Live Data & Requests

> *The radio is the heartbeat of the organization, so its code follows one rule above all others: audio comes first. Every visual feature degrades gracefully, and nothing a visual feature does may stop the stream — especially on an iPhone with the screen locked.*

8.1 Station configuration lives in code, on purpose

The Radio.co station ID and URLs are infrastructure, not content. If they lived in Sanity, a CMS outage, a rate limit or an accidentally cleared field would silence the station. They are defined once, at module scope, in `BEARCAST_RADIO`:

SCRIPTS.JS · LINES 236-245 — SCRIPTS.JS — THE STATION

```
236 window.BEARCAST_RADIO = window.BEARCAST_RADIO || (function(){
237   const STATION_ID = 'sa4dd72cde';
238   const ORIGINAL_TITLE = document.title;
239   return {
240     STATION_ID,
241     STREAM_URL:  `https://s4.radio.co/${STATION_ID}/listen`,
242     STATUS_URL:  `https://public.radio.co/stations/${STATION_ID}/status`,
243     HISTORY_URL: `https://public.radio.co/stations/${STATION_ID}/history`,
244     POLL_MS: 12000,
245     ORIGINAL_TITLE,
```

Both players (home hero and Radio page) are a plain `<audio id="radio-audio" preload="none">` element pointed at `STREAM_URL` on play and cleared on pause, so a paused tab holds no connection. Now-playing data comes from Radio.co's public status endpoint every 12 seconds (paused while the tab is hidden unless audio is playing, so the lock screen stays current), and the Radio page also reads the history endpoint for Recently Played.

8.2 Parsing tracks and spotting live DJ sets

Radio.co reports the current song as one `"Artist - Title"` string; `parseTrack` splits on the first `" - "` and drops file extensions. A subtler problem: when a DJ goes live, broadcast software often sends no track info, so Radio.co keeps reporting the last automated song — sometimes for an hour. `isDjSetUnknownTrack` distrusts the reported song when it is the one that was playing just before the DJ went live, or when it claims to have been playing for over 15 minutes.

SCRIPTS.JS · LINES 268-286 — BEARCAST_RADIO — STALE-TRACK DETECTION DURING A LIVE SET

```
268 isDjSetUnknownTrack(s){
269   const live = !!(s && s.status === 'online' && s.source && s.source.type === 'live');
270   const ct = (s && s.current_track) || {};
271   const title = ct.title || '';
272   if (!live) {
273     this._lastAutoTitle = title;
274     this._preLiveTitle = null;
275     this._wasLive = false;
276     return false;
```

```
  }
  if (!this._wasLive) {
    this._wasLive = true;
    this._preLiveTitle = this._lastAutoTitle || null;
  }
  if (!title) return true;
  if (this._preLiveTitle && title === this._preLiveTitle) return true;
  const started = Date.parse(ct.start_time);
  return !isNaN(started) && Date.now() - started > 15 * 60e3;
},
```

Instead of a generic “Live DJ” line, the site then names the scheduled show from the Sanity schedule, handling slots that run past midnight:

SCRIPTS.JS · LINES 297-312 — BEARCAST_RADIO — THE SHOW ON AIR NOW

```
showOnAir(){
  const days = ['sun', 'mon', 'tue', 'wed', 'thu', 'fri', 'sat'];
  const d = new Date(), now = d.getHours() * 60 + d.getMinutes();
  const today = days[d.getDay()], yesterday = days[(d.getDay() + 6) % 7];
  const hm = t => { const m = String(t || '').match(/^(\d{1,2}):(\d{2})$/); return m ? +m[1] * 60 + +m[2] :
null; };
  for (const sh of this.shows) {
    for (const sl of (sh && sh.timeSlots) || []) {
      const st = hm(sl.startTime), en = hm(sl.endTime);
      if (st == null || en == null) continue;
      const overnight = en <= st;
      if (sl.day === today && now >= st && (overnight || now < en)) return sh;
      if (overnight && sl.day === yesterday && now < en) return sh;
    }
  }
  return null;
},
```

The Live DJ badge deliberately ignores Radio.co’s `source.collaborator.name`: that is whichever Radio.co account is connected, not necessarily the person on the mic.

8.3 Staying on the air: reconnect with back-off

Live streams drop on network blips (Wi-Fi to cellular, a Radio.co edge restart). While the listener still wants audio, the player retries after 1, 2, 4, 8, 15 and 30 seconds, treats 12 seconds of buffering as a drop, retries immediately when the browser reports it is back online, and gives up cleanly after about a minute so the UI never sits on a dead “LIVE”.

SCRIPTS.JS · LINES 445-489 — BEARCAST_RADIO — AUTO-RECONNECT

```
attachReconnect(audio, opts){
  if (!audio) return;
  const { isActive, onGiveUp } = opts || {};
  const DELAYS = [1000, 2000, 4000, 8000, 15000, 30000];
  const STALL_MS = 12000; // buffering this long = treat as dropped
  const STREAM_URL = this.STREAM_URL;
  const setState = (st) => this.setConnectionState(st);
  let attempt = 0, retryTimer = null, stallTimer = null;

  const clearTimers = () => {
    clearTimeout(retryTimer); retryTimer = null;
    clearTimeout(stallTimer); stallTimer = null;
```

```
457   };
458   const retryNow = () => {
459     retryTimer = null;
460     if (!isActive()) return;
461     audio.src = STREAM_URL;
462     audio.play().catch((err) => {
463       // Autoplay blocked (no user gesture) — retrying won't help.
464       if (err && err.name === 'NotAllowedError') giveUp();
465       // Anything else surfaces as an 'error' event → schedule() again.
466     });
467   };
468   const giveUp = () => {
469     clearTimers();
470     attempt = 0;
471     if (isActive() && onGiveUp) onGiveUp();
472     setState('error');
473   };
474   const schedule = () => {
475     if (!isActive() || retryTimer) return;
476     clearTimeout(stallTimer); stallTimer = null;
477     if (attempt >= DELAYS.length) { giveUp(); return; }
478     setState('reconnecting');
479     retryTimer = setTimeout(retryNow, DELAYS[attempt++]);
480   };
481
482   audio.addEventListener('playing', () => { attempt = 0; clearTimers(); });
483   audio.addEventListener('error',   () => { if (isActive()) schedule(); });
484   audio.addEventListener('ended',   () => { if (isActive()) schedule(); });
485   const armStall = () => {
486     if (!isActive() || stallTimer || retryTimer) return;
487     stallTimer = setTimeout(() => { stallTimer = null; schedule(); }, STALL_MS);
488   };
489   audio.addEventListener('waiting', armStall);
```

The same object sets the tab title ("♪ Artist — Title · Bearcast Radio") and Media Session metadata for lock screens, Control Center and media keys, explicitly disabling seek and skip buttons a live stream cannot honour.

8.4 The audio tap: real visuals without risking the stream

The waveform (home) and spectrum, VU meter and visualizer styles (Radio) can react to the real audio through the Web Audio API. That is risky: once a media element is routed into Web Audio it cannot be unplugged, iOS can suspend a Web Audio graph when the screen locks (silencing the radio), and analysis needs the stream to be served with CORS headers. `bcCreateAudioTap` wraps all of that in safety nets:

1. iPhone and iPad never route audio through Web Audio; they get stylized visuals. A device whose audio context gets stuck in the background is remembered (`bc_viz_off`) and also gets stylized visuals.
2. The stream is loaded in CORS mode and connected to the analyser only once it is actually playing and the context is running. If it fails to load in CORS mode, it instantly retries as a normal stream and remembers that for a day.
3. Analysis happens *before* the gain node, so visuals ignore the volume slider; volume is applied on the gain node.

SCRIPTS.JS · LINES 2320-2342 — BCCREATEAUDIOTAP — CONNECTING ONLY WHEN IT IS SAFE

```
2320  function connect(){
2321    if (src || !ctx || !isActive()) return;
2322    if (ctx.state !== 'running') return; // never connect into a silent graph
```

```
  try {
    src = ctx.createMediaElementSource(audio);
    analyser = ctx.createAnalyser();
    analyser.fftSize = 4096;
    analyser.smoothingTimeConstant = 0.78;
    analyser.minDecibels = -90;
    analyser.maxDecibels = -18;
    gain = ctx.createGain();
    src.connect(analyser);           // analyse before volume, so visuals ignore the volume
    src.connect(gain);
    gain.connect(ctx.destination);
    setVolume(outVol);
    live = true;
    freq = new Uint8Array(analyser.frequencyBinCount);
    wave = new Uint8Array(analyser.fftSize);
  } catch (_) {
    wanted = false;
  }
  onChange();
}
```

SCRIPTS.JS · LINES 2352-2365 — BCCREATEAUDIOTAP — CORS FAILURE FALLS BACK TO PLAIN PLAYBACK

```
audio.addEventListener('error', e => {
  if (!pending || !isActive()) return;
  // Couldn't load the stream in CORS mode: play it normally right away.
  // Could be a real CORS problem or a network blip on first play — block
  // only briefly so a blip doesn't cost long-term real visuals.
  pending = false;
  wanted = false;
  if (navigator.onLine !== false) block('cors', 1);
  e.stopImmediatePropagation(); // don't let auto-reconnect retry in CORS mode
  audio.removeAttribute('crossorigin');
  audio.src = STREAM_URL;
  audio.play().catch(() => {});
  onChange();
});
```

The spectrum maps 4096-point FFT bins onto log-spaced bands from 20 Hz to 20 kHz with a slight treble lift; the VU meter converts RMS level to decibels. A silence watchdog on the Radio page switches to stylized mode if the analyser only ever sees perfect digital silence. Phones draw at 30 fps instead of 60.

SCRIPTS.JS · LINES 2384-2399 — BCCREATEAUDIOTAP — LOG-SPACED BANDS

```
function freqValues(n){
  if (!live || !analyser) return null;
  analyser.getByteFrequencyData(freq);
  const binHz = ctx.sampleRate / analyser.fftSize;
  const out = new Array(n);
  for (let i = 0; i < n; i++) {
    const f0 = 20 * Math.pow(1000, i / n), f1 = 20 * Math.pow(1000, (i + 1) / n);
    const b0 = Math.min(freq.length - 1, Math.floor(f0 / binHz));
    const b1 = Math.min(freq.length - 1, Math.max(b0, Math.floor(f1 / binHz)));
    let m = 0;
    for (let b = b0; b <= b1; b++) if (freq[b] > m) m = freq[b];
    const x = i / n;
    out[i] = Math.min(1, Math.pow(m / 255, 1.3) * (0.85 + 0.5 * x)); // lift the treble a little
  }
  return out;
}
```

8.5 Live listener count

bearcastmedia.com is the only place the stream plays, so the site counts its own listeners. While a player is actually playing, the browser sends a heartbeat to `/api/listeners` about once a minute with a random per-tab ID (no name, account or IP is stored), and a "stopped" note when sound stops or the tab closes — via `sendBeacon`, which survives the page unloading. `timeupdate` doubles as the heartbeat because it keeps firing in background tabs where timers are throttled.

SCRIPTS.JS · LINES 559-568 — BC_LISTENERS — WHAT TRIGGERS A HEARTBEAT

```
559 function track(audio){
560   if (!audio || audio.__bcListen) return;
561   audio.__bcListen = true;
562   audio.addEventListener('playing', start);
563   // timeupdate keeps firing while sound plays, even in a background tab
564   // where timers can be slowed down, so it doubles as the heartbeat.
565   audio.addEventListener('timeupdate', () => { if (!audio.paused) { if (on) beat(false); else start(); } });
566   ['pause', 'ended', 'emptied', 'error'].forEach(ev => audio.addEventListener(ev, stop));
567   window.addEventListener('pagehide', stop);
568 }
```

The server side (a D1 table and a daily peak) is described in §12.3. The Radio page shows "12 listening now · Today's peak: 23"; the home mini player shows 🎧 12. Without the database binding the endpoint answers `{"listeners": null}` and the count is simply hidden.

8.6 Recently Played and Apple Music links

Each recent track gets Spotify and Apple Music buttons. On iPhone, an Apple Music *search* link opens the app but ignores the search term, so each song is looked up once through Apple's public iTunes Search API and its link replaced with the song's own page. Matching is strict — exact title plus matching artist, with remixes, live and acoustic versions penalised unless that is what played — and only `https://music.apple.com/` URLs are ever accepted. Lookups run one at a time, 400 ms apart, results are cached 30 days (misses 7) in `bc_am_links`, and a link the visitor is about to tap jumps to the front of the queue.

SCRIPTS.JS · LINES 5111-5128 — RADIO MODULE — CHOOSING THE RIGHT APPLE MUSIC MATCH

```
5111 function bcAmPick(results, artist, title){
5112   const t = bcAmNorm(title), a = bcAmNorm(bcPrimaryArtist(artist));
5113   const variant = /\b(remix|live|acoustic|instrumental|karaoke|version|edit|mix|demo|sped up|slowed)\b/;
5114   let best = null, bestScore = -1;
5115   (results || []).forEach(r => {
5116     if (!r || typeof r.trackViewUrl !== 'string' || !/^https:\/\/music\.apple\.com\//.test(r.trackViewUrl))
     return;
5117     const rt = bcAmNorm(r.trackName), ra = bcAmNorm(r.artistName);
5118     let sc;
5119     if (rt === t) sc = 6;
5120     else if (rt.startsWith(t) || t.startsWith(rt)) sc = 3;
5121     else return;
5122     if (a && ra && (ra === a || ra.includes(a) || a.includes(ra))) sc += 4;
5123     else if (a) sc -= 4;
5124     if (variant.test(rt) && !variant.test(t)) sc -= 3;
5125     if (sc > bestScore) { bestScore = sc; best = r; }
5126   });
5127   return best && bestScore >= 6 ? best.trackViewUrl.replace(/[?&]uo=\d+/, '') : '';
5128 }
```

8.7 Song requests

The request form posts to `/api/song-request` (§12.2), which validates it and forwards it to the DJs' Slack channel. Success is shown only after the server confirms delivery; on failure the fields are kept so the listener can just press the button again, with friendly messages for the 30-second throttle (429) and invalid input (400). A hidden `website` field acts as a honeypot. Listeners can opt in to sending their Your Bearcast tier so DJs can give a shout-out — the browser sends only a tier *index* and star count; the label is looked up on the server.

SCRIPTS.JS · LINES 7607-7611 — RADIO MODULE — THE REQUEST PAYLOAD

```
7607  res = await fetch('/api/song-request', {
7608    method: 'POST',
7609    headers: { 'Content-Type': 'application/json' },
7610    body: JSON.stringify({ song, name, message: msg, website, listener: bcRequestTier() })
7611  });
```

After a successful request the song text is remembered for 12 hours (`bc_pending_requests`). When a track that started after the request matches it as whole words ("under pressure queen", "Mr. Brightside - The Killers"), the page celebrates and awards the **Called It** badge.

8.8 Opt-in notifications

"Notify me when a DJ goes live" asks for notification permission only when clicked, remembers the choice in `bc_dj_notify`, and keeps polling in a hidden tab while enabled. Mobile Chrome forbids `new Notification()`, so the code falls back to the service worker's `showNotification`.

8.9 Schedule, shows and the homepage widget

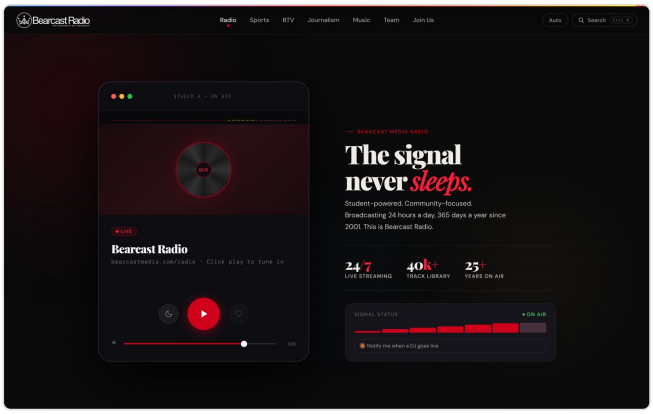
Radio shows come from Sanity on both pages. The homepage converts each show's slots into today's list ("6:00 AM – 9:00 AM"), computes the current show and its progress (wrapping midnight), and refreshes every minute; the Radio page groups slots by day for the weekly tabs. Schedule rows and "Our Shows" cards (with genre filter and sort) open the same details pop-up with hosts, all weekly slots and a "Listen live" button. The schedule uses the visitor's local clock, which is correct for a station serving one community: a 6 AM show is 6 AM in Cincinnati, and daylight saving is handled by the browser.

SCRIPTS.JS · LINES 3497-3525 — HOME MODULE — SANITY SHOWS → TODAY'S SCHEDULE

```
3497  function bcShowsToTodaySchedule(shows){
3498    if (!Array.isArray(shows)) return null;
3499    const dayKeys = ['sun','mon','tue','wed','thu','fri','sat'];
3500    const todayKey = dayKeys[new Date().getDay()];
3501    const items = [];
3502    for (const sh of shows) {
3503      const slots = Array.isArray(sh.timeSlots) ? sh.timeSlots : [];
3504      for (const slot of slots) {
3505        if (slot.day !== todayKey) continue;
3506        const startMins = _bcParseHHMM(slot.startTime);
3507        const endMins = _bcParseHHMM(slot.endTime);
3508        const hostName = sh.hostDisplayName
3509          || (Array.isArray(sh.hostNames) && sh.hostNames.length
3510              ? sh.hostNames.join(' & ')
3511              : 'Bearcast Crew');
3512        items.push({
3513          time: _bcMinsToLabel(startMins),
```

```
        end: _bcMinsToLabel(endMins),
        show: sh.name || 'Untitled Show',
        host: hostName,
        genre: sh.genre || 'music',
        _startMins: startMins, // for sorting only
      });
    }
  }
  items.sort((a,b) => a._startMins - b._startMins);
  items.forEach(i => delete i._startMins);
  return items;
}
```

RADIO PAGE · DARK



RADIO PAGE · LIGHT

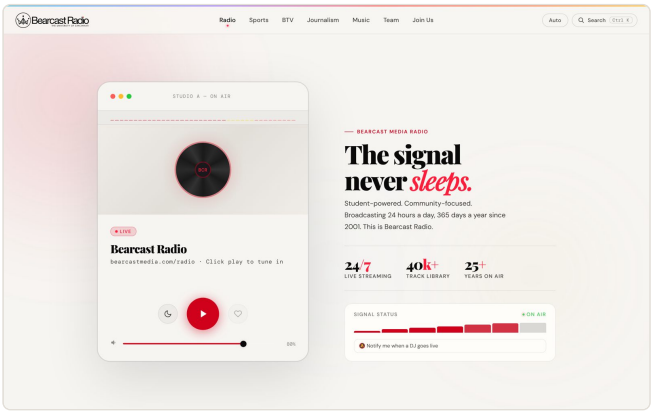


Figure 8.1 — The Radio page rendered from the current source in both themes (Radio.co and Sanity unreachable in the test environment, so the signal panel shows its idle state).

§ 09 — LISTENER FEATURES

Your Bearcast — Personal Stats Without Accounts

> *Wrapped-style recaps, streaks, listener tiers, badges, liked songs and a year-end Wrapped — computed entirely from a listening log in the visitor's own browser. Nothing leaves the device unless the listener exports or shares it.*

9.1 Where listening is counted

The shared module `BC_ME` owns the data so both players (home and Radio) count into the same history. Pages report three things: a second of real listening (`tick` , only while the audio element is actually playing, on a plain `setInterval` so background listening counts), a song heard (`logHeard` , once per song and start time), and likes (`toggleLike`). Saving to `localStorage` is batched every 15 seconds and on `pagehide` / `visibilitychange` .

SCRIPTS.JS · LINES 2719-2731 — BC_ME — ONE SECOND OF LISTENING

```
// One second of real listening, bucketed by day, hour and month.
function tick(){
  const now = new Date();
  const k = bcDayKey(now);
  const day = bcMe.days[k] || (bcMe.days[k] = { s: 0, h: {} });
  day.s++;
  day.h[now.getHours()] = (day.h[now.getHours()] || 0) + 1;
  bcMonthAgg(bcMonthKey(now)).s++;
  bcMe.total++;
  tickCount++;
  if (tickCount % 15 === 0) bcMeSave();
  return tickCount;
}
```

9.2 The data model

SCRIPTS.JS · LINES 5498-5511 — RADIO MODULE — THE SHAPE OF BC_ME_V1

```
// bcMe shape (v2):
//   days    { 'YYYY-MM-DD': { s: secs, h: { hour: secs } } }    kept ~400 days
//   plays   [{ t, a, art, at, live }]                           kept 60 days
//   months  { 'YYYY-MM': { s, songs, live, art: { artistKey: n },
//             trk: { songKey: { t, a, c } } } }                 kept forever (compact)
//   artists { artistKey: { n: displayName, c: plays, f: firstHeardMs } }
//   total   all-time seconds listened
//   pruned  seconds from days old enough to have been dropped from `days`
//   liked   [{ t, a, art, at }]
//   badges  { badgeId: earnedAtMs }
//   best    longest streak ever (days)
//   tier, stars  last tier index / prestige stars we celebrated
//   profile { name, emoji, theme }
//   lastKey last logged track, so reloads don't double-count a song
```

Recent detail is kept short and long-term facts are kept compact: individual plays for 60 days (max 1,500), per-day/per-hour seconds for about 400 days (so a year-end Wrapped always has the whole year), and per-month aggregates forever — trimmed to the top 40 artists and songs once a month is over. Seconds from pruned days move into `pruned` so all-time totals stay verifiable.

9.3 Tiers, unlocks and badges

Ten listener tiers are based purely on all-time listening. Each unlocks avatar emoji, and five unlock a player and share-card theme with its own "signature" touches; past Hall of Fame, a prestige ★ is earned every 50 hours.

SCRIPTS.JS · LINES 5596-5608 — RADIO MODULE — TIERS (MIN = MINUTES)

```
5596 const BC_TIERS = [
5597   { name: 'Static',          icon: '📻', min: 0,    ring: 'red',    avatars: ['🐻', '🎧', '🔥', '🌙'] },
5598   { name: 'Tuning In',       icon: '🎚️', min: 30,   ring: 'red',    avatars: ['🐸', '🍓'] },
5599   { name: 'Rookie',          icon: '🌱', min: 120,  ring: 'red',    avatars: ['🦉', '🛹'] },
5600   { name: 'Regular',         icon: '🎧', min: 300,  ring: 'bronze', avatars: ['🎸', '💿'], theme:
       'midnight' },
5601   { name: 'Superfan',        icon: '⭐', min: 600,  ring: 'bronze', avatars: ['🦋', '⚡'], theme:
       'emerald' },
5602   { name: 'Resident',        icon: '🏠', min: 1200, ring: 'silver', avatars: ['🌈', '👽'] },
5603   { name: 'Headliner',       icon: '🎤', min: 2400, ring: 'silver', avatars: ['🦄', '🪩'], theme: 'gold' },
5604   { name: 'Station Icon',    icon: '💎', min: 4200, ring: 'gold',   avatars: ['🐉', '🚀'], theme: 'neon' },
5605   { name: 'Bearcast Legend', icon: '👑', min: 6000, ring: 'gold',   avatars: ['💎'],       theme:
       'royal' },
5606   { name: 'Hall of Fame',    icon: '🏆', min: 9000, ring: 'holo',   avatars: ['👑'],       theme:
       'aurora' },
5607 ];
5608 const BC_PRESTIGE_MIN = 3000; // past Hall of Fame, a ★ every 50 hours
```

BADGE	EARNED WHEN	BADGE	EARNED WHEN
🎧 First Listen	a minute of listening	🔥 On a Roll	3-day streak
⏳ Hour of Power	an hour in one day	💯 Locked In	7-day streak
🦉 Night Owl	listening midnight–4 AM	💿 Crate Digger	25 different artists
🌅 Early Bird	tuned in before 8 AM	💖 Curator	10 saved songs
🎙️ Live & Direct	caught a live DJ show	📣 Called It	your request made it on air

A day counts toward a streak after one minute of listening, and a streak is not "broken" in the morning before you press play — it counts back from yesterday if today has no listening yet. On first load, existing progress sets the tier silently; only later level-ups get a celebration.

SCRIPTS.JS · LINES 5562-5569 — RADIO MODULE — STREAKS

```
5562 function bcStreak(){
5563   const d = new Date();
5564   if (!bcListenedOn(d)) d.setDate(d.getDate() - 1);
5565   let n = 0;
5566   while (bcListenedOn(d)) { n++; d.setDate(d.getDate() - 1); }
5567   if (n > bcMe.best) bcMe.best = n;
5568   return n;
5569 }
```

9.4 Keeping the numbers possible: tamper checks

All of this lives in the listener's browser, so anyone with developer tools can edit it; only a server could truly verify listening. What the code *can* guarantee is that the numbers are possible. `bcSanitize` runs on load, on cross-tab sync and on import. It removes anything before launch day (`BC_HISTORY_START = '2026-10-07'`) or in the future, caps hours at 3,600 seconds and days at 24 hours, requires every total to be backed by day records, limits a month to one song per 30 seconds, and rejects keys like `__proto__` that could pollute JavaScript prototypes. On import, any problem rejects the whole file.

SCRIPTS.JS · LINES 2539-2569 — BC_ME — BCSANITIZE (EXCERPT)

```
2539  // Days: the source of truth. Each hour ≤ 1 h, each day ≤ 24 h and ≤ its hours.
2540  let daySum = 0;
2541  Object.keys(m.days).forEach(k => {
2542    if (!/^\d{4}-\d{2}-\d{2}$/.test(k) || k < BC_HISTORY_START || k > maxKey) { delete m.days[k]; issues++;
      return; }
2543    const d = bcObj(m.days[k]), h = {};
2544    let hourSum = 0;
2545    Object.keys(bcObj(d.h)).forEach(hr => {
2546      const n = bcNum(d.h[hr]);
2547      if (!/^\d{1,2}$/.test(hr) || +hr > 23 || !n) return;
2548      if (n > 3600) issues++;
2549      h[+hr] = Math.min(n, 3600);
2550      hourSum += h[+hr];
2551    });
2552    const s = Math.min(bcNum(d.s), hourSum, 86400);
2553    if (bcNum(d.s) > s + 60) issues++;
2554    m.days[k] = { s, h };
2555    daySum += s;
2556  });
2557
2558  // Days older than ~400 are pruned and their seconds kept in `pruned`,
2559  // which can't exceed 24 h for every day between launch and the oldest kept day.
2560  const dayKeys = Object.keys(m.days).sort();
2561  const oldest = dayKeys[0] || bcDayKey();
2562  const maxPruned = Math.max(0, Math.round((bcKeyToDate(oldest) - startMs) / 864e5)) * 86400;
2563  if (bcNum(m.pruned) > maxPruned) issues++;
2564  m.pruned = Math.min(bcNum(m.pruned), maxPruned);
2565
2566  // All-time listening (what tiers use) must be backed by the day records.
2567  const maxTotal = daySum + m.pruned;
2568  if (bcNum(m.total) > maxTotal + 120) issues++;
2569  m.total = Math.min(bcNum(m.total), maxTotal);
```

Tiers sent with song requests are labelled *self-reported* in Slack for exactly this reason (§12.2).

9.5 Other tabs, other devices

A `storage` event fires when another tab saves. If this tab is not playing it adopts the other tab's copy (so an unlike there sticks here); if it is playing it merges, taking the larger value of every counter so nothing is double-counted. Export downloads `bearcast-history-YYYY-MM-DD.json` ; import on another device merges the same way, so importing the same file twice changes nothing.

SCRIPTS.JS · LINES 6825-6840 — RADIO MODULE — IMPORT IS VALIDATED BEFORE IT IS MERGED

```
6825 if (youImport) {
6826   youImport.addEventListener('change', async () => {
6827     const file = youImport.files && youImport.files[0];
6828     youImport.value = '';
6829     if (!file) return;
6830     if (file.size > 5 * 1024 * 1024) { window.bcToast('That file is too big to be a Bearcast export.', {
      type: 'error' }); return; }
6831     try {
6832       const json = JSON.parse(await file.text());
6833       if (!json || json.app !== 'bearcast-radio' || !json.data) throw new Error('not ours');
6834       const src = bcMeNormalize(json.data);
6835       if (bcSanitize(src)) {
6836         window.bcToast('This history file has been edited or damaged, so it can't be imported.', { type:
      'error', duration: 5000 });
6837         return;
6838       }
6839       const before = bcMe.total;
6840       bcMergeHistory(src);
```

9.6 Recaps, Wrapped and share cards

Recaps cover this week, this month, this year or a past calendar year: minutes, songs, artists, top five artists, top song (only if heard twice), discoveries, longest streak and a “listening persona” from the hours you listen. The year-end **Wrapped** opens December 1 – January 15 for the year that is ending, with a September–November teaser; `?wrapped=2026` previews any year the feature could have data for.

SCRIPTS.JS · LINES 6858-6867 — RADIO MODULE — WHEN WRAPPED IS OPEN

```
6858 function bcWrappedWindow(){
6859   const q = window.bcGetDeepLink && window.bcGetDeepLink('wrapped');
6860   // Preview links only for years the feature could have data for.
6861   if (q && /^\d{4}$/.test(q) && +q >= 2026 && +q <= new Date().getFullYear()) return { year: +q, open:
      true };
6862   const now = new Date();
6863   if (now.getMonth() === 11) return { year: now.getFullYear(), open: true };
6864   if (now.getMonth() === 0 && now.getDate() <= 15) return { year: now.getFullYear() - 1, open: true };
6865   if (now.getMonth() >= 8) return { year: now.getFullYear(), open: false }; // Sep-Nov teaser
6866   return null;
6867 }
```

Share cards are 1080×1920 PNGs drawn on a canvas in the chosen theme. On phones they go straight to the system share sheet (`navigator.share` with a file); elsewhere the PNG downloads. The rendered card is cached so a second tap after a share-permission error shares instantly.

§ 10 — VIDEO

BTV: YouTube as the Source of Truth

> *Every video on youtube.com/@BearcastMedia appears on the BTV page automatically, with live view counts. The Studio is now optional: it only adds details YouTube cannot.*

10.1 Listing the channel — /api/btv-videos

A Pages Function lists the channel. By default it reads YouTube's public RSS feed (no setup, newest 15 videos, no durations). If a `YOUTUBE_API_KEY` secret is configured it uses the YouTube Data API instead: the channel's uploads playlist (the channel ID with `UC` → `UU`), paged 50 at a time up to 200 videos, then one statistics request per 50 for view counts, durations and live-stream start times. If the API fails, it falls back to RSS. Responses are cached at the edge for 15 minutes, so YouTube is asked at most a few times an hour however busy the site is.

FUNCTIONS/API/BTV-VIDEOS.JS · LINES 30-55 — BTV-VIDEOS.JS — TWO SOURCES, ONE EDGE CACHE

```
30 export async function onRequestGet(context) {
31   const cache = caches.default;
32   const cacheKey = new Request(new URL('/api/btv-videos?v=1', context.request.url).toString());
33   const cached = await cache.match(cacheKey);
34   if (cached) return cached;
35
36   let videos = [];
37   let source = 'rss';
38   const key = context.env && context.env.YOUTUBE_API_KEY;
39   if (key) {
40     try { videos = await fromDataApi(key); source = 'api'; }
41     catch (e) { console.warn('[btv-videos] Data API failed, using RSS:', e && e.message); videos = []; }
42   }
43   if (!videos.length) {
44     try { videos = await fromRss(); source = 'rss'; }
45     catch (e) {
46       console.error('[btv-videos] RSS failed:', e && e.message);
47       return json({ channel: CHANNEL_ID, source: 'none', videos: [] }, 502, 'no-store');
48     }
49   }
50
51   const res = json({ channel: CHANNEL_ID, source, updated: new Date().toISOString(), videos }, 200,
52     `public, max-age=${CACHE_SECONDS}`);
53   if (videos.length) context.waitUntil(cache.put(cacheKey, res.clone()));
54   return res;
55 }
```

FUNCTIONS/API/BTV-VIDEOS.JS · LINES 154-164 — BTV-VIDEOS.JS — EVERY FIELD IS BOUNDED AND TYPED

```
function video(id, title, desc, published, views, runtime) {
  return {
    id,
    title: String(title || 'Untitled').slice(0, 300),
    desc: String(desc || '').slice(0, 2000),
    published: published && !isNaN(Date.parse(published)) ? new Date(published).toISOString() : null,
    views: typeof views === 'number' && isFinite(views) ? views : null,
    runtime: runtime || null,
    thumb: `https://i.ytimg.com/vi/${id}/hqdefault.jpg`
  };
}
```

10.2 Merging Studio details

`BC_BTV.load()` fetches the video list and the Studio's `episode` documents in parallel and merges them by YouTube ID: Studio fields win (title, show, season/episode, description, featured), YouTube supplies view counts and fills in a missing date or runtime. Episodes with no matching video still appear. Anything published in the last 14 days is marked NEW automatically.

SCRIPTS.JS · LINES 1451-1469 — BC_BTV — MERGE RULE

```
function merge(sanityDocs, videos){
  const byYt = new Map();
  const out = videos.map(fromYouTube);
  out.forEach(e => byYt.set(e.ytId, e));
  sanityDocs.map(fromSanity).forEach(s => {
    const yt = s.ytId && byYt.get(s.ytId);
    if (yt) {
      // Studio fields win; YouTube supplies views (and date/runtime if missing).
      Object.assign(yt, s, { id: yt.id, views: yt.views, ts: s.ts || yt.ts, runtime: s.runtime || yt.runtime,
        desc: s.desc || yt.desc, thumb: yt.thumb });
    } else out.push(s);
  });
  const now = Date.now();
  out.forEach(e => {
    e.date = fmtDate(e.ts);
    e.isNew = !!(e.isNewFlag || (e.ts && now - e.ts < 14 * 864e5));
  });
  return out.sort((a, b) => b.ts - a.ts);
}
```

YouTube videos have no category, so `classify` infers one from the title: "Queen City Sessions" and "Mini City Session" become music sessions, titles with "vs." or a sport name become game broadcasts ("audio only" noted), "news" or "report" becomes News, everything else is a BTV show. The same classification feeds the Music page's **Music Videos** shelf.

SCRIPTS.JS · LINES 1420-1430 — BC_BTV — INFERRING A SHOW TYPE

```
function classify(title){
  const t = String(title || '');
  const session = t.match(/\b((?:Queen|Mini) City Sessions?)\b/i);
  if (session) return { show: 'music', showName: session[1].replace(/\bsessions?\b/i, 'Session'), emoji: '🎸' };
  if (/\bsessions?\b|\blive performance\b|\bmusic video\b|\bofficial video\b|\bacoustic\b/i.test(t)) return { show: 'music', showName: 'Music', emoji: '🎵' };
  if (/\b(vs\.?|versus|@)\s|\b(football|basketball|volleyball|soccer|baseball|softball|hockey|tennis|lacrosse)\b/i.test(t)) {
    return { show: 'sports', showName: /audio only/i.test(t) ? 'Game Broadcast · Audio' : 'Game Broadcast', emoji: '🏆' };
  }
  if (/\bnews\b|\breport\b/i.test(t)) return { show: 'news', showName: 'News', emoji: '📡' };
  return { show: 'show', showName: 'BTV', emoji: '📺' };
}
```

10.3 Playing a video

Videos open in the shared dialog using the privacy-enhanced `youtube-nocookie.com` embed, built only from an ID that has passed the 11-character check. The dialog carries a “More on BTV” deep link (`btv.html?ep=yt-…`) and an “Open in YouTube” link.

SCRIPTS.JS · LINES 1497-1518 — BC_BTV — THE EMBED IS BUILT FROM A VALIDATED ID

```
function openVideo(ep){
  if (!ep) return;
  const esc = window.bcEscHtml;
  const ytId = ID_RE.test(ep.ytId || '') ? ep.ytId : '';
  const views = fmtViews(ep.views);
  const meta = [ep.showName, ep.date, views ? views + ' views' : '', ep.runtime].filter(Boolean).map(esc).join(' · ');
  window.bcDialog.open({
    label: ep.title, wide: true, video: !!ytId, accent: 'var(--violet,#845EF7)',
    html:
      (ytId ? '<div class="bc-dlg-video"><iframe src="https://www.youtube-nocookie.com/embed/' + ytId +
        '?rel=0&modestbranding=1&autoplay=1" title="' + esc(ep.title) +
'" allow="autoplay; encrypted-media; picture-in-picture; fullscreen" allowfullscreen></iframe></div>' : '') +
… 9 lines omitted …
  });
}
```

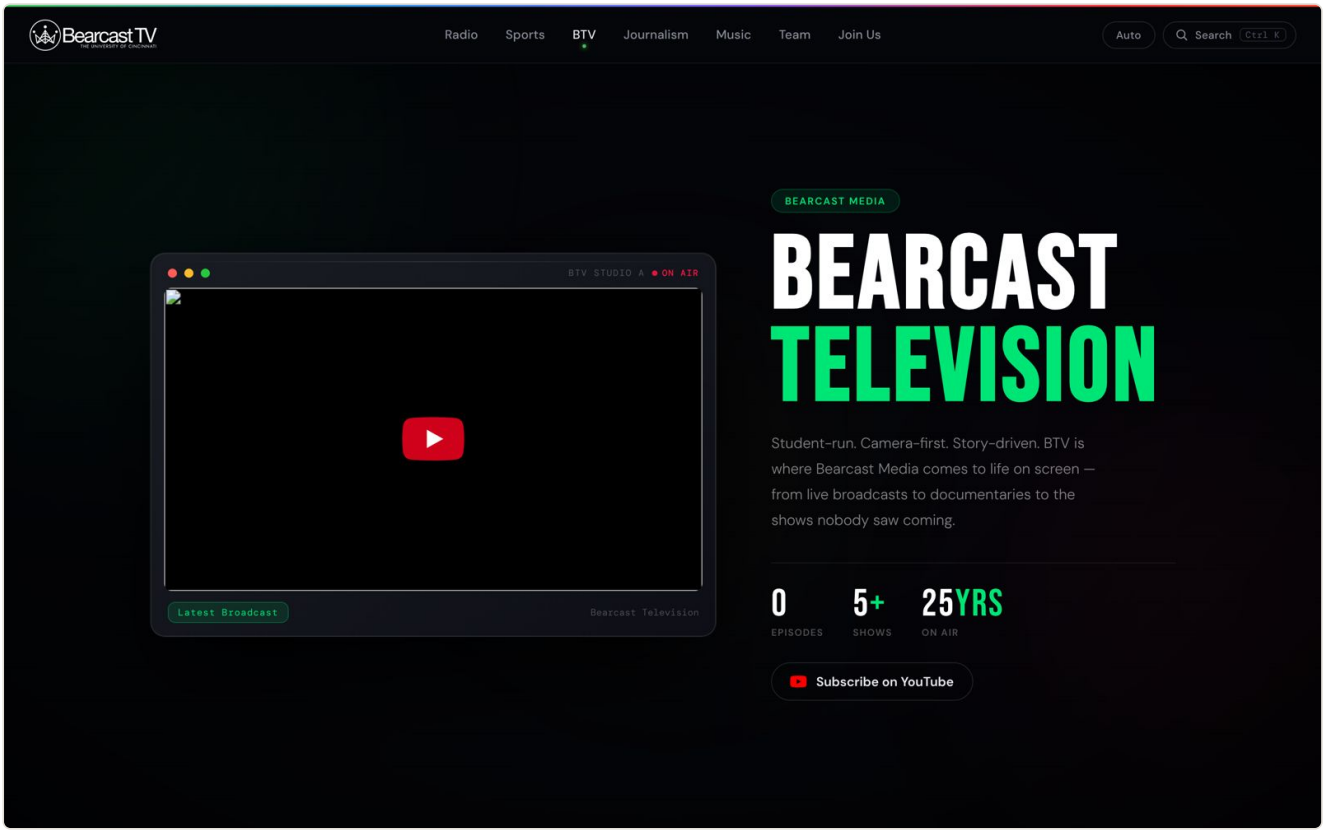


Figure 10.1 — BTV rendered locally from the current source (no network to YouTube in the test environment, so the counter shows 0 episodes). Bebas Neue gives the page its broadcast-title voice.

§ 11 — PEOPLE & RECRUITING

Team, GBMs & the Interest Form

11.1 Directors in seniority order

The Team page loads every `person` with **Is a Director** ticked, together with the shows they host and the stories they have written — joined inside the same GROQ query with `references()` and a reverse author lookup. Photos are cropped to their hotspot and only accepted from `cdn.sanity.io`.

SCRIPTS.JS · LINES 10007-10020 — TEAM MODULE — ONE QUERY FOR PEOPLE, SHOWS AND BYLINES

```
const SANITY_DIRECTORS_QUERY = `
  *[_type == "person" && isDirector == true]
  | order(name asc)
  {
    name, role, dept, bio, tags, term, isExecutive, accentColor, pronouns,
    "shows": *[_type == "radioShow" && references(^._id)].name,
    "stories": *[_type in ["article","review","coverage"] && author._ref == ^._id]{_type, desk},
    "photo": photo{
      alt, crop, hotspot,
      "url": asset->url,
      "dims": asset->metadata.dimensions{width, height}
    }
  }
`;
```

The page reads like an org chart rather than A–Z: the executive board first, then departments in a fixed order, and within each group by position, inferred from keywords in the free-text **Role / Title**. A department's own head ("Radio Director" in Radio) outranks other directors there. Editors therefore control ordering simply by writing clear titles.

SCRIPTS.JS · LINES 9911-9920 — TEAM MODULE — RANK FROM A TITLE

```
const DEPT_ORDER = ['Executive', 'Radio', 'BTV', 'Journalism', 'Music', 'Sports'];
function bcRoleRank(title){
  const t = String(title || '').toLowerCase();
  if (/\b(general manager|station manager|president|editor[- ]in[- ]chief|executive director)\b/.test(t)
&& !/\b(vice|assistant|associate)\b/.test(t)) return 0;
  if (/\b(vice president|vice-president|deputy|managing editor|managing director)\b/.test(t)) return 1;
  if (/\b(assistant|associate|asst\.?)\b/.test(t)) return 5;
  if (/\b(director|editor|head|chief)\b/.test(t)) return 2;
  if (/\b(manager|coordinator|lead)\b/.test(t)) return 3;
  return 4;
}
```

SCRIPTS.JS · LINES 9928-9936 — TEAM MODULE — THE FULL ORDERING

```
function bcBySeniority(a, b){
  const ea = a.exec ? 0 : 1, eb = b.exec ? 0 : 1;
  if (ea !== eb) return ea - eb;
  const da = DEPT_ORDER.indexOf(a.dept), db = DEPT_ORDER.indexOf(b.dept);
  if (da !== db) return (da < 0 ? 99 : da) - (db < 0 ? 99 : db);
  const ra = bcRankIn(a), rb = bcRankIn(b);
  if (ra !== rb) return ra - rb;
  return (a.name || '').localeCompare(b.name || '', undefined, {sensitivity: 'base'});
}
```

11.2 General Body Meetings

The Join page lists upcoming GBMs from Sanity (cancelled ones are filtered out in GROQ), shows the next meeting's location wherever the page mentions where GBMs are held (default Emery 3225), and counts down to the real next meeting. Meeting times are entered as free text ("6:30 PM", "7 PM", "19:00") and parsed; the countdown target is computed in **Eastern time** regardless of the visitor's time zone, so an out-of-town visitor sees the correct moment.

SCRIPTS.JS · LINES 10066-10076 — JOIN MODULE — 6:30 PM IN CINCINNATI, AS AN ABSOLUTE TIME

```
function bcEasternTime(y, mo, d, h, min){
  const guess = Date.UTC(y, mo, d, h, min);
  try {
    const parts = {};
    new Intl.DateTimeFormat('en-US', { timeZone: 'America/New_York', hourCycle: 'h23',
      year: 'numeric', month: 'numeric', day: 'numeric', hour: 'numeric', minute: 'numeric' })
      .formatToParts(new Date(guess)).forEach(p => { parts[p.type] = +p.value; });
    const asEastern = Date.UTC(parts.year, parts.month - 1, parts.day, parts.hour % 24, parts.minute);
    return new Date(guess + (guess - asEastern));
  } catch (_) { return new Date(y, mo, d, h, min); }
}
```

Until Sanity answers, the page renders a computed every-other-Thursday fallback so it is never empty. The homepage hero shows the same next meeting in one small combined request with the newest article, review and game coverage (§5.1).

11.3 The interest form

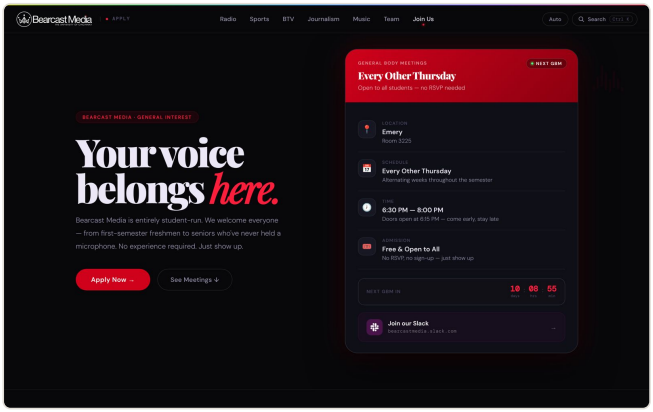
The application form validates in the browser (names, a sane email, year, at least one interest area, "why Bearcast"), caps lengths, and posts JSON to `/api/join`, which validates everything again and delivers it to the directors' **private** Slack channel (§12.2). Success is shown only when the server confirms delivery; specific errors ("give it a minute", "check your email address", "one answer is too long") map from the server's error codes. Applications are not stored on the website.

SCRIPTS.JS · LINES 10319-10347 — JOIN MODULE — SUBMIT AND REACT TO THE SERVER

```
const payload={
  fname, lname, email, year, why,
  major: val('major'), experience: val('experience'), howheard: val('howheard'),
  depts: deptChecked, website: honey ? honey.value : ''
};
const reset=()=>{btn.disabled=false;btn.textContent='Submit Application →';};
const ctrl=new AbortController();
```

```
const timer=setTimeout(()=>ctrl.abort(),15000);
fetch('/api/join',{
  method:'POST',
  headers:{'Content-Type':'application/json'},
  body:JSON.stringify(payload),
  signal:ctrl.signal
}).then(r=>r.json().catch(()=>({})).then(d=>({status:r.status,d}))).then(({status,d})=>{
  if(status===200&&d&&d.ok){
    document.getElementById('applicationForm').style.display='none';
    document.getElementById('formSuccess').classList.add('show');
    return;
  }
  const err=d&&d.error;
  if(err==='slow_down') window.bcToast('You just submitted — give it a minute before sending again.',
{type:'error'});
  else if(err==='email_invalid') {shake('email');window.bcToast('Please double-check your email address.',
{type:'error'});}
  else if(err==='too_long') window.bcToast('One of your answers is too long — please shorten it a little.',
{type:'error'});
  else window.bcToast('Sorry — your application didn\u2019t send. Please try again in a moment.',
{type:'error',duration:6000});
  reset();
}).catch(()=>{
  window.bcToast('Couldn\u2019t reach Bearcast — check your connection and try again.',
{type:'error',duration:6000});
  reset();
}).finally(()=>clearTimeout(timer));
```

JOIN



TEAM

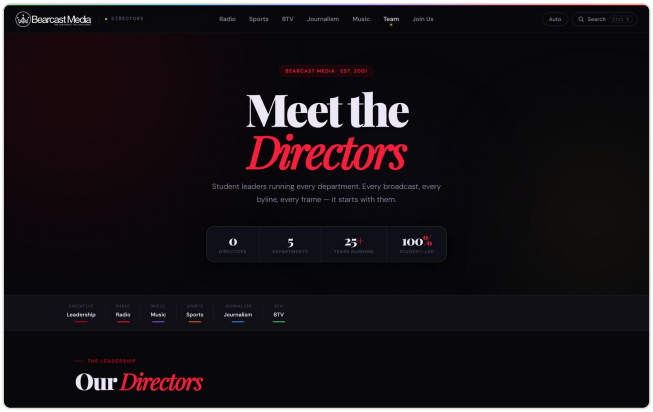


Figure 11.1 — Join and Team rendered locally from the current source. The GBM card shows the computed fallback schedule and live countdown; the Team page shows its loading state with Sanity unreachable.

§ 12 — EDGE FUNCTIONS

Cloudflare Pages Functions

> *The site is static, except where it cannot be: keeping secrets, describing a story to link-preview bots, counting listeners and listing YouTube. Five small functions do exactly that, and nothing else runs on a server.*

12.1 Scoping: _routes.json

By default a Pages project with a `functions/` folder routes every request through the Functions runtime. `_routes.json` limits it to the paths that need code; everything else is served straight from the CDN without touching a function.

_ROUTES.JSON · LINES 1-12 — _ROUTES.JSON

```
{
  "version": 1,
  "include": [
    "/api/*",
    "/sitemap.xml",
    "/journalism", "/journalism.html",
    "/music", "/music.html",
    "/sports", "/sports.html",
    "/btv", "/btv.html"
  ],
  "exclude": []
}
```

PATH	FILE	METHODS	PURPOSE
`/api/song-request`	`api/song-request.js`	POST (else 405)	Validate → DJs' Slack channel
`/api/join`	`api/join.js`	POST (else 405)	Validate → directors' private Slack channel
`/api/listeners`	`api/listeners.js`	GET, POST	Live count & daily peak in D1
`/api/btv-videos`	`api/btv-videos.js`	GET, HEAD	YouTube channel list (§10)
article pages, `/sitemap.xml`, `/api/*`	`_middleware.js`	all	Link previews, JSON-LD, sitemap, security headers

12.2 Forms to Slack

Both form endpoints follow the same five-step pattern: only the site's own origin may post; a honeypot field earns bots a fake "ok"; bodies are capped before parsing (8 KB / 16 KB) and every field is length-limited or allow-listed; one submission per IP per 30 s (requests) or 60 s (applications); and all user text is escaped for Slack's mrkdwn so nobody can trigger `@channel` pings or inject links. Webhook URLs are Cloudflare secrets (`SLACK_SONG_REQUEST_WEBHOOK`, `SLACK_JOIN_WEBHOOK`) and never appear in the repository or the browser. If the secret is missing the endpoint answers 503 instead of crashing; if Slack fails, the throttle entry is removed so the person can retry.

FUNCTIONS/API/SONG-REQUEST.JS · LINES 83-120 — SONG-REQUEST.JS — THE GATEKEEPING, IN ORDER

```
83  export async function onRequestPost({ request, env }){
84    if (!originAllowed(request)) return json({ ok: false, error: 'forbidden' }, 403);
85
86    const webhook = env.SLACK_SONG_REQUEST_WEBHOOK;
87    if (!webhook) {
88      console.error('[song-request] SLACK_SONG_REQUEST_WEBHOOK is not set');
89      return json({ ok: false, error: 'not_configured' }, 503);
90    }
91
92    // A real request is well under 4 KB; refuse anything big before parsing it.
93    const MAX_BODY = 8 * 1024;
94    if (Number(request.headers.get('Content-Length') || 0) > MAX_BODY) {
95      return json({ ok: false, error: 'too_large' }, 413);
96    }
97    let data;
98    try {
99      const raw = await request.text();
100     if (raw.length > MAX_BODY) return json({ ok: false, error: 'too_large' }, 413);
101     data = JSON.parse(raw);
102   } catch (_) {
103     return json({ ok: false, error: 'bad_request' }, 400);
104   }
105
106   if (!data || typeof data !== 'object' || Array.isArray(data)) return json({ ok: false, error:
    'bad_request' }, 400);
107
108   // Honeypot — pretend it worked so the bot moves on.
109   if (clean(data.website)) return json({ ok: true }, 200);
110
111   const song    = clean(data.song);
112   const name    = clean(data.name);
113   const message = clean(data.message);
114
115   if (!song) return json({ ok: false, error: 'song_required' }, 400);
116   if (song.length > LIMITS.song || name.length > LIMITS.name || message.length > LIMITS.message) {
117     return json({ ok: false, error: 'too_long' }, 400);
118   }
119
120   const ip = request.headers.get('CF-Connecting-IP') || 'unknown';
```

FUNCTIONS/API/SONG-REQUEST.JS · LINES 66-68 — SONG-REQUEST.JS — SLACK-SAFE TEXT

```
66  function slackEsc(s){
67    return String(s).replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
68  }
```

FUNCTIONS/API/SONG-REQUEST.JS · LINES 36-46 — SONG-REQUEST.JS — THE TIER LABEL COMES FROM THE SERVER, NOT THE BROWSER

```
const TIERS = [
  '📻 Static', '🎚 Tuning In', '🌱 Rookie', '🎧 Regular', '⭐ Superfan',
  '🏠 Resident', '🎤 Headliner', '💎 Station Icon', '👑 Bearcast Legend', '🏆 Hall of Fame'
];
function tierLabel(listener){
  if (!listener || typeof listener !== 'object') return '';
  const i = listener.tier, stars = listener.stars;
  if (!Number.isInteger(i) || i < 0 || i >= TIERS.length) return '';
  const s = Number.isInteger(stars) && stars > 0 && stars <= 99 && i === TIERS.length - 1 ? ` ★${stars}` :
'';
  return TIERS[i] + s;
}
```

The interest form adds stricter rules because it carries personal data: the year and “how did you hear” must come from fixed lists, the email must match a pattern that also excludes `<`, `>` and `|` (characters that could break Slack’s `<mailto:…|…>` link), and Slack’s 3,000-character section limit is respected.

FUNCTIONS/API/JOIN.JS · LINES 92-108 — JOIN.JS — VALIDATION

```
const first = clean(data.fname), last = clean(data.lname);
const email = clean(data.email).toLowerCase();
const year = clean(data.year), major = clean(data.major);
const experience = clean(data.experience), why = clean(data.why);
const howHeard = HOW_HEARD.includes(clean(data.howheard)) ? clean(data.howheard) : '';
const depts = (Array.isArray(data.depts) ? data.depts : [])
  .map(clean).filter(Boolean).slice(0, LIMITS.depts).map(d => d.slice(0, LIMITS.dept));

if (!first) return json({ ok: false, error: 'name_required' }, 400);
if (!EMAIL_RE.test(email)) return json({ ok: false, error: 'email_invalid' }, 400);
if (!YEARS.includes(year)) return json({ ok: false, error: 'year_required' }, 400);
if (!depts.length) return json({ ok: false, error: 'dept_required' }, 400);
if (!why) return json({ ok: false, error: 'why_required' }, 400);
if (first.length > LIMITS.name || last.length > LIMITS.name || email.length > LIMITS.email ||
    major.length > LIMITS.major || experience.length > LIMITS.experience || why.length > LIMITS.why) {
  return json({ ok: false, error: 'too_long' }, 400);
}
```

12.3 Listener count on D1

`/api/listeners` stores one row per listening tab in a Cloudflare D1 (SQLite) database bound as `LISTENERS_DB`. Tables are created on first use. A listener counts until 2.5 minutes after their last heartbeat; GET responses are shared for 15 seconds from the edge cache so a busy Radio page does not hit the database per viewer. All SQL uses bound parameters. Two cost-conscious details: there is deliberately *no* index on `seen` (D1 bills each index update as an extra written row and the table only holds a few dozen rows), and the daily peak is written only when it actually rises. The free tier’s 100,000 writes a day covers about 65 people listening around the clock.

FUNCTIONS/API/LISTENERS.JS · LINES 81-91 — LISTENERS.JS — COUNT AND RAISE TODAY'S PEAK

```
81  async function countAndPeak(db){
82    const now = Date.now(), day = stationDay();
83    const row = await db.prepare('SELECT COUNT(*) AS n FROM listeners WHERE seen > ?').bind(now -
    WINDOW_MS).first();
84    const n = (row && row.n) || 0;
85    await db.batch([
86      db.prepare('INSERT OR IGNORE INTO listener_peaks (day, peak) VALUES (?, ?)').bind(day, n),
87      db.prepare('UPDATE listener_peaks SET peak = ? WHERE day = ? AND peak < ?').bind(n, day, n),
88    ]);
89    const p = await db.prepare('SELECT peak FROM listener_peaks WHERE day = ?').bind(day).first();
90    return { listeners: n, peak: Math.max(n, (p && p.peak) || 0), day };
91  }
```

FUNCTIONS/API/LISTENERS.JS · LINES 48-57 — LISTENERS.JS — SENDBEACON MAY OMIT ORIGIN, SO FALL BACK TO SEC-FETCH-SITE

```
48  function originAllowed(request){
49    // Only our own pages may post here: the browser's Origin must match the
50    // host this request was sent to (bearcastmedia.com, www, a preview deploy
51    // or local wrangler all match themselves; any other site doesn't).
52    const origin = request.headers.get('Origin') || '';
53    // navigator.sendBeacon from our own page may omit Origin on some browsers.
54    if (!origin) return request.headers.get('Sec-Fetch-Site') === 'same-origin';
55    if (ALLOWED_ORIGINS.includes(origin)) return true;
56    try { return new URL(origin).host === new URL(request.url).host; } catch (_) { return false; }
57  }
```

12.4 Link previews, JSON-LD and the sitemap

A shared link such as `bearcastmedia.com/journalism?article=some-story` is the same HTML for everyone; the story is loaded by JavaScript. iMessage, Instagram, Slack, Discord and X do not run JavaScript, so they used to show the generic page title. The middleware looks the story up in Sanity (or, for `?ep=yt-…`, YouTube's oEmbed) with a 2.5-second budget and rewrites the page's `<title>`, description, Open Graph and Twitter tags and canonical URL with Cloudflare's streaming `HTMLRewriter`, then appends schema.org JSON-LD — `NewsArticle`, `Review` (with album, artist and rating) or `VideoObject`. If anything is slow or fails, the page is served unchanged: the feature can skip a nicer preview but never break the site.

FUNCTIONS/_MIDDLEWARE.JS · LINES 48-68 — _MIDDLEWARE.JS — THE REQUEST PATH

```
48  export async function onRequest(context) {
49    const url = new URL(context.request.url);
50    if (url.pathname.startsWith('/api/')) return apiHeaders(await context.next());
51    if (url.pathname === '/sitemap.xml') return sitemap(context);
52
53    const page = url.pathname.replace(/^\/|\.html$/g, '');
54    const param = PAGES[page];
55    const res = await context.next();
56    const isHtml = (res.headers.get('content-type') || '').includes('text/html');
57    if (!param || !isHtml || !res.ok || context.request.method !== 'GET') return secure(res, isHtml);
58
59    const key = (url.searchParams.get(param) || '').slice(0, 200);
60    if (!key) return secure(res, true);
61
62    let meta = null;
63    try { meta = await withTimeout(page === 'btv' ? episodeMeta(key) : storyMeta(key), 2500); }
64    catch (e) { console.warn('[middleware] preview lookup failed:', e && e.message); }
65    if (!meta) return secure(res, true);
```

```
  meta.url = SITE + '/' + page + '?' + param + '=' + encodeURIComponent(key);
  return secure(rewrite(res, meta), true);
}
```

FUNCTIONS/_MIDDLEWARE.JS · LINES 201-203 — _MIDDLEWARE.JS — JSON-LD AND ATTRIBUTE VALUES ARE ESCAPED

```
const ld = Object.assign({ '@context': 'https://schema.org', mainEntityOfPage: meta.url, url: meta.url },
meta.ld);
const ldJson = JSON.stringify(ld).replace(/</g, '\\u003c').replace(/>/g, '\\u003e').replace(/&/g, '\
\u0026');
const esc = s => String(s).replace(/&/g, '&amp;').replace(/"/g, '&quot;').replace(/</g, '&lt;').replace(/>/
g, '&gt;');
```

Social images are cropped from the editor's image to 1200×630 respecting crop and hotspot, falling back to the 25th-anniversary card (`imgs/og-image.jpg`). `/sitemap.xml` lists the eight pages plus every published story with its last-modified date (the article cards are JavaScript, so crawlers could not otherwise find them). It is cached at the edge for an hour, but only when Sanity actually returned stories. `robots.txt` points to it.

The middleware has one more job: Cloudflare's `_headers` file does not apply to responses that pass through a Function, so it re-applies the full security header set to those HTML pages and a JSON-appropriate subset to `/api/*`. A pull-request check keeps the two copies of the CSP identical (§14.1).

FUNCTIONS/_MIDDLEWARE.JS · LINES 70-77 — _MIDDLEWARE.JS — HEADERS FOR FUNCTION RESPONSES

```
function secure(res, isHtml) {
  const out = new Response(res.body, res);
  if (isHtml) {
    for (const k in SECURITY_HEADERS) if (!out.headers.has(k)) out.headers.set(k, SECURITY_HEADERS[k]);
    out.headers.set('Cache-Control', 'public, max-age=0, must-revalidate');
  }
  return out;
}
```

§ 13 — DEPLOYMENT

Build, Deployment & the Offline Layer

> *Pushing to `main` deploys. Cloudflare Pages runs one shell script that turns the two files developers edit into small per-page bundles, stamps a version on every asset URL and removes everything that is not the website.*

13.1 Cloudflare Pages settings

Production branch	`main` → bearcastmedia.com (also bearcastmedia.pages.dev)
Build command	`bash build.sh`
Build output directory	repository root
Secrets	`SLACK_SONG_REQUEST_WEBHOOK`, `SLACK_JOIN_WEBHOOK`, optional `YOUTUBE_API_KEY`
Bindings	D1 database as `LISTENERS_DB` (optional; count hidden without it)
Preview deploys	every branch and pull request gets its own URL — the development policy is to test there before merging
Rollback	Deployments → any earlier deploy → Rollback

13.2 build.sh, step by step

The script refuses to run anywhere but Cloudflare (`CF_PAGES=1`) because it edits and deletes files — run on a laptop it would delete the Studio source from the working copy.

BUILD.SH · LINES 34-41 — BUILD.SH — GUARD

```
34  set -euo pipefail
35  cd "$(dirname "$0")"
36
37  if [ "${CF_PAGES:-}" != "1" ]; then
38    echo "build.sh: this only runs as the Cloudflare Pages build step (it edits and deletes files)." >&2
39    echo "          Nothing was changed. You don't need to run it locally." >&2
40    exit 1
41  fi
```

Step 0 — per-page bundles, then minify. `split-assets.js` writes `css/<page>.css` and `js/<page>.js` and rewrites each page's links; esbuild (pinned to an exact version) minifies them. If esbuild cannot be fetched, the readable bundles ship rather than failing the deploy.

BUILD.SH · LINES 47-59 — BUILD.SH — BUNDLES AND MINIFICATION

```
47  node .github/scripts/split-assets.js
48
49  # Minify the bundles (≈35% smaller JS, ≈12% smaller CSS after compression).
50  # Your styles.css / scripts.js stay readable — only the deployed copies are
51  # minified. esbuild is pinned; if it can't be fetched the build still ships
52  # the readable bundles rather than failing.
53  ESBUILD="esbuild@0.28.2"
54  if npx --yes "$ESBUILD" js/*.js --minify --log-level=warning --outdir=js --allow-overwrite \
55     && npx --yes "$ESBUILD" css/*.css --minify --log-level=warning --outdir=css --allow-overwrite; then
56    echo "build.sh: minified css/ and js/ with $ESBUILD"
57  else
58    echo "build.sh: WARNING — minify skipped ($ESBUILD unavailable); deploying readable bundles" >&2
59  fi
```

Step 1 — version stamping. Every page links `styles.css?v=__ASSET_VERSION__` and `scripts.js?v=__ASSET_VERSION__`, and `sw.js` names its caches with the same placeholder. The build replaces it with the first eight characters of the commit SHA, so each deploy gets brand-new asset URLs — browsers can never pair new HTML with old cached JavaScript, and the service worker clears its old caches automatically. The build fails if no placeholder is found or one is left behind.

BUILD.SH · LINES 61-81 — BUILD.SH — STAMPING THE ASSET VERSION

```
61  # Commit ID from Cloudflare Pages; timestamp if building elsewhere.
62  V="${CF_PAGES_COMMIT_SHA:-}"
63  V="${V:0:8}"
64  [ -n "$V" ] || V="$(date +%Y%m%d%H%M%S)"
65
66  files=(*.html sw.js)
67  count=$( (grep -o "__ASSET_VERSION__" "${files[@]}" || true) | wc -l | tr -d ' ')
68  if [ "$count" -eq 0 ]; then
69    echo "build.sh: no __ASSET_VERSION__ placeholders found — refusing to deploy unversioned assets" >&2
70    exit 1
71  fi
72
73  # -i.bak works with both GNU (Cloudflare) and BSD (macOS) sed.
74  sed -i.bak "s/__ASSET_VERSION__/${V}/g" "${files[@]}"
75  rm -f ./*.html.bak sw.js.bak
76
77  if grep -q "__ASSET_VERSION__" "${files[@]}"; then
78    echo "build.sh: a placeholder was left behind" >&2
79    exit 1
80  fi
81  echo "build.sh: stamped ${count} placeholder(s) with asset version ${V}"
```

Step 2 — remove non-site files. The output directory is the repository root, so without this step every committed file (README, Studio source, workflows) would be downloadable from bearcastmedia.com. The weekly health check verifies that `/README.md` returns 404.

BUILD.SH · LINES 84-106 — BUILD.SH — STRIP EVERYTHING THAT IS NOT THE WEBSITE

```
84  NOT_SITE=(
85    bearcast-studio      # Sanity Studio source (deployed separately via Sanity)
86    ios                  # native app source, if ever added to this repo
87    README.md
88    LICENSE
89    package.json
90    package-lock.json
```

```
  .gitignore
  .github
  .vscode
  .claude
  node_modules
)
for path in "${NOT_SITE[@]}"; do
  if [ -e "$path" ]; then rm -rf -- "$path"; echo "build.sh: removed from deploy: $path"; fi
done
# Any other stray docs / TypeScript at the root (the site has none).
for f in ./*.md ./*.ts; do
  [ -e "$f" ] && rm -f -- "$f" && echo "build.sh: removed from deploy: ${f#./}"
done
# Last: this script itself (safe — bash has already read it).
rm -f -- "$0"
echo "build.sh: removed from deploy: build.sh"
```

13.3 split-assets.js — why it cannot change behaviour

Every page used to download the full 946 KB of CSS and JavaScript. The splitter keeps, for each page, all shared rules and code plus that page's own `body[data-page="…"]` rules and `//#page` module, in original order so the cascade is identical. It is safe by construction: a rule written for another page could never match, and another page's module returns on its first line anyway. Anything it cannot be certain about — a page selector inside `:not()`, `:is()`, `:has()` — stays on every page. It uses no npm packages, because Cloudflare's build image has only plain Node.

.GITHUB/SCRIPTS/SPLIT-ASSETS.JS · LINES 106-110 — SPLIT-ASSETS.JS — WHICH PAGE A SELECTOR BELONGS TO

```
function selectorPage(sel){
  const outer = sel.replace(/:(not|is|where|has|matches)\((?:[^()]|\([^()]*\))*\)/g, '');
  const m = outer.match(/\bbody[^\s>+~]*\[data-page=["']?([\w-]+)["']?\]/);
  return m ? m[1] : null;
}
```

.GITHUB/SCRIPTS/SPLIT-ASSETS.JS · LINES 129-140 — SPLIT-ASSETS.JS — CUTTING SCRIPTS.JS AT //#PAGE MARKERS

```
function splitJs(src){
  const lines = src.split('\n');
  const shared = []; const modules = {}; let cur = null;
  for (const line of lines) {
    const start = line.match(/^\/\/#page ([\w-]+)/);
    if (start) { if (cur) throw new Error('scripts.js: //#page ' + start[1] + ' before //#endpage of ' + cur); cur = start[1]; modules[cur] = modules[cur] || []; shared.push(`\u0000${cur}`); continue; }
    if (/^\/\/#endpage/.test(line)) { if (!cur) throw new Error('scripts.js: //#endpage without //#page'); cur = null; continue; }
    (cur ? modules[cur] : shared).push(line);
  }
  if (cur) throw new Error('scripts.js: //#page ' + cur + ' is never closed with //#endpage');
  return page => shared.map(l => l.startsWith('\u0000') ? (l.slice(1) === page ? modules[page].join('\n') : '') : l).join('\n');
}
```

The splitter also points the service worker's precache at the offline page's bundles. Measured on this snapshot (`node .github/scripts/split-assets.js --dry`, before minification):

```
$ node .github/scripts/split-assets.js --dry
split-assets: per-page bundles (was 946 KB CSS+JS on every page)
```

```
404.html          css/404.css          72 KB   js/404.js          174 KB
btv.html          css/btv.css         102 KB   js/btv.js          186 KB
index.html        css/home.css        119 KB   js/home.js         203 KB
join.html         css/join.css         99 KB   js/join.js         188 KB
journalism.html   css/journalism.css   98 KB   js/journalism.js   195 KB
music.html        css/music.css       103 KB   js/music.js        206 KB
radio.html        css/radio.css       183 KB   js/radio.js        350 KB
sports.html       css/sports.css      103 KB   js/sports.js       205 KB
team.html         css/team.css         86 KB   js/team.js         190 KB
```

Every page except Radio now downloads about 250–320 KB of readable CSS+JS instead of 946 KB, before minification (≈35% smaller JS, ≈12% smaller CSS once compressed) and Brotli at the edge.

13.4 Caching headers

Because asset URLs change with every deploy, CSS and JS can be cached for a year with `immutable` — no revalidation round-trip on any page view. HTML is always revalidated so deploys propagate immediately. The service worker itself must never be cached by the HTTP cache, or browsers could keep running an old worker for up to 24 hours.

_HEADERS · LINES 44-66 — _HEADERS — CACHING RULES

```
44  ## Every page links styles.css / scripts.js as ?v=<deploy id> (stamped by
45  ## build.sh), so each deploy gets a brand-new URL. That makes it safe to cache
46  ## them for a year with no revalidation round trip on every page view.
47  /*.css
48    Cache-Control: public, max-age=31536000, immutable
49  /*.js
50    Cache-Control: public, max-age=31536000, immutable
51
52  ## …except the service worker, which must always be fetched fresh, otherwise
53  ## browsers keep running the old SW for up to 24 hours after a deploy.
54  ## "! Cache-Control" drops the /*.js value above before setting this one.
55  /sw.js
56    ! Cache-Control
57    Cache-Control: no-cache
58  /imgs/*
59    Cache-Control: public, max-age=604800, stale-while-revalidate=2592000
60  ## Font files never change in place (a new font gets a new file name).
61  /fonts/*
62    Cache-Control: public, max-age=31536000, immutable
63
64  ## HTML pages: short cache + always revalidate so deploys propagate quickly.
65  /*.html
66    Cache-Control: public, max-age=0, must-revalidate
```

13.5 The service worker

`sw.js` is registered on every page after load. Its strategies are chosen so caching is invisible where freshness matters:

REQUEST	STRATEGY
HTML pages	Network first; cache the response; offline → the cached page, else the offline page

REQUEST	STRATEGY
Versioned CSS/JS (`?v=`)	Cache first — the URL changes every deploy, so a cached copy is never stale
Unversioned CSS/JS	Network first (revalidating past the HTTP cache), cache fallback
Images and fonts	Stale-while-revalidate
Sanity, Radio.co, YouTube; any other origin; any non-GET	Never intercepted

SW.JS · LINES 32-58 — SW.JS — INSTALL-TIME PRECACHE AND NETWORK-ONLY HOSTS

```
32 const PRECACHE_ASSETS = [
33   // The exact URLs the pages request (build.sh stamps the version).
34   '/styles.css?v=__ASSET_VERSION__',
35   '/scripts.js?v=__ASSET_VERSION__',
36   '/imgs/favicon.svg',
37   '/imgs/favicon-32.png',
38   '/imgs/favicon-16.png',
39   '/imgs/apple-touch-icon.png',
40   '/imgs/favicon.ico',
41   // Nav logos (both themes) so the header still renders offline.
42   '/imgs/logo-bearcast-media-black.webp',
43   '/imgs/logo-bearcast-media-white.webp'
44 ];
45
46 // The offline fallback — the one page that must be cached before the
47 // visitor can ever need it, so it goes in at install time.
48 const PRECACHE_PAGES = ['/404.html'];
49
50 // Hosts the SW must never serve from cache: live data and streams.
51 const NETWORK_ONLY_HOSTS = [
52   'apicdn.sanity.io',
53   'cdn.sanity.io',
54   'public.radio.co',
55   'radio.co',
56   'youtube.com',
57   'youtube-nocookie.com'
58 ];
```

Install now precaches only the offline page (404.html and its bundles), the icons and the nav logos. Pages are cached as they are visited, so pages a visitor has opened keep working offline and anything else shows the offline page, which detects `navigator.onLine === false` and says so (“You’re offline. This page hasn’t been saved yet.”). Pre-downloading every page’s bundle would not be worth a visitor’s data plan.

SW.JS · LINES 62-68 — SW.JS — A REAL-WORLD FIX

```
62 // Cloudflare redirects /page.html → /page, so precached pages are
63 // stored as *redirected* responses — and browsers refuse to show one for a
64 // navigation (a blank ERR_FAILED page instead of the offline page). Serve a
65 // plain copy instead.
66 function unredirect(res){
67   return res && res.redirected ? new Response(res.body, { status: res.status, statusText: res.statusText,
   headers: res.headers }) : res;
68 }
```

Cloudflare redirects `/radio.html` to `/radio`, so a cached page can be a *redirected* response — and browsers refuse to show one for a navigation (a blank ERR_FAILED instead of the offline page). `unredirect()` serves a plain copy.

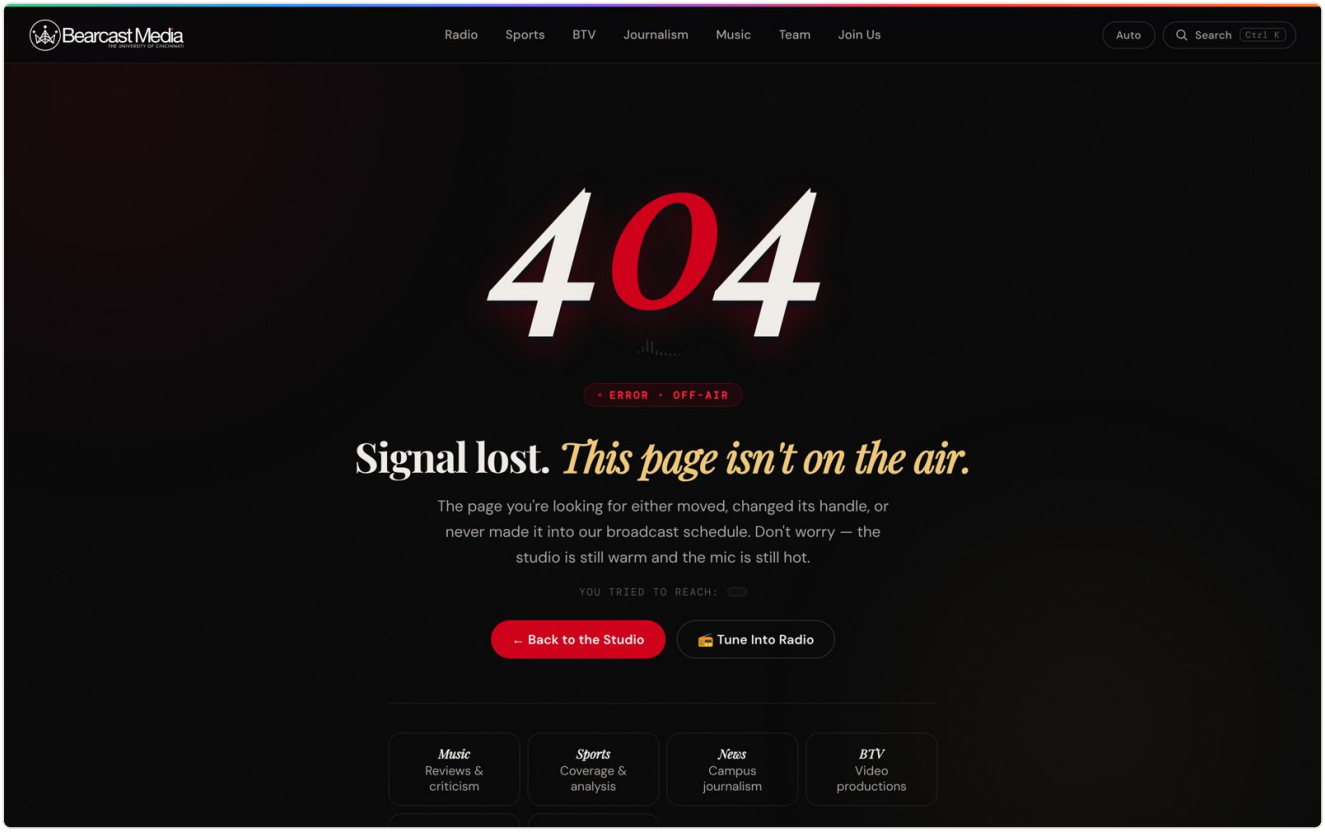


Figure 13.1 — The 404 page, which is also the offline page. Its small inline script (the second hash in the CSP) suggests the closest real page for mistyped addresses using edit distance, and runs even if `scripts.js` fails to load.

13.6 The Studio deploys separately

From `bearcast-studio/`, `npm run deploy` publishes to bearcast.sanity.studio (Node 22.12+). The website build never touches it, and `build.sh` removes its source from the site deploy.

§ 14 — AUTOMATION

Automation, Quality Gates & Local Development

> *The site is built to run for long stretches without code changes. GitHub Actions stand guard: every pull request is checked, the live site and everything it depends on is tested every Monday, and the CMS is backed up every Sunday.*

14.1 Pull-request checks

`.github/workflows/pr-checks.yml` runs two jobs on every pull request to `main`; the branch ruleset requires both to pass. **Site checks** runs `.github/scripts/pr-checks.sh` (safe to run locally — it only reads files) and then ESLint with correctness rules only (undefined variables, unreachable code, duplicate keys). **Studio build** installs the pinned Studio dependencies with `npm ci` and runs `sanity build`, so a broken schema cannot be merged.

#	CHECK	CATCHES
1	JavaScript parses	`scripts.js`, `sw.js` and every function (as ES modules)
2	JSON config valid	`_routes.json`
3	Asset placeholders	every page links both assets with `?v=__ASSET_VERSION__`; `sw.js` has its placeholder
4	Local files exist	every `src` / `href` / `poster`, CSS `url()` and `imgs/` path in JS
5	No test code	temporary tier-testing switches left in
6	CSP strict and in sync	`_headers` and the middleware CSP differ; `'unsafe-inline'` in `script-src`; any inline handler; any inline script whose SHA-256 is not in the policy
7	Bundles build	`split-assets.js --dry` (e.g. an unclosed `//#page` block)

.GITHUB/SCRIPTS/PR-CHECKS.SH · LINES 80-102 — PR-CHECKS.SH — CHECK 6 COMPUTES EACH INLINE SCRIPT'S HASH

```
80  echo "6 · Content-Security-Policy is strict and in sync"
81  csp_out=$(node - <<'JS'
82  const fs = require('fs'), crypto = require('crypto');
83  const out = [];
84  const headers = (fs.readFileSync('_headers', 'utf8').match(/Content-Security-Policy: (.*)/) || [])[1] || '';
85  const mw = (fs.readFileSync('functions/_middleware.js', 'utf8').match(/'Content-Security-Policy':
    "([^"]*)"/) || [])[1] || '';
86  if (!headers) out.push('BAD _headers has no Content-Security-Policy');
87  if (headers !== mw) out.push('BAD the CSP in _headers and functions/_middleware.js differ — keep them
    identical');
88  if (/script-src[^;]*'unsafe-inline'/.test(headers)) out.push("BAD script-src must not allow 'unsafe-
    inline'");
89  for (const f of fs.readdirSync('.').filter(f => f.endsWith('.html'))) {
90    const html = fs.readFileSync(f, 'utf8');
91    const handler = html.match(/<[^>]+\son[a-z]+\s*=\s*["'][^>]*>/i);
92    if (handler) out.push(`BAD ${f}: inline event handler (use data-act="…" instead): ${handler[0].slice(0,
    80)}`);
```

```
  for (const m of html.matchAll(/<script(?![^>]*\bsrc=)(?![^>]*type="application\/ld\+json")[^>]*>([\s\S]*?)<\/script>/g)) {
    const h = "'sha256-" + crypto.createHash('sha256').update(m[1]).digest('base64') + "'";
    if (!headers.includes(h)) out.push(`BAD ${f}: an inline <script> isn't allowed by the CSP — if you edited it, put ${h} in script-src (in _headers and _middleware.js)`);
  }
}
console.log(out.join('\n'));
JS
)
if [ -z "$csp_out" ]; then ok "one strict CSP, no inline handlers, inline scripts match their hashes"
else while IFS= read -r line; do bad "${line#BAD }"; done <<< "$csp_out"; fi
```

Run against this snapshot, all seven checks pass and ESLint reports no problems:

```
$ bash .github/scripts/pr-checks.sh
1 · JavaScript parses           ✅ scripts.js  ✅ sw.js  ✅ 5 functions
2 · JSON config is valid        ✅ _routes.json
3 · Asset-version placeholders  ✅ 9 pages + sw.js
4 · Local files exist           ✅ every local image, page, stylesheet and script exists
5 · No temporary test code      ✅ none found
6 · CSP strict and in sync      ✅ one strict CSP, no inline handlers, inline scripts match their hashes
7 · Per-page bundles build      ✅ css/<page>.css and js/<page>.js build for every page
All site checks passed.
$ npx eslint scripts.js sw.js functions      0 problems
```

14.2 Weekly site health check

Every Monday at 13:00 UTC (after Sunday's backup), `health-check.sh` tests what the site depends on, read-only. If Cloudflare's bot protection blocks GitHub's data-centre runners from the custom domain — Cloudflare working as intended — it tests the identical deployment on `bearcastmedia.pages.dev`. Any failure opens (or comments on) one GitHub issue labelled `health-check`, which GitHub emails to the maintainer; the issue closes itself when everything passes again.

AREA	WHAT IS TESTED
Website	8 pages return 200, version placeholder stamped, reference the scripts; CSP, HSTS, X-Frame-Options, nosniff present; `/README.md` is 404
Functions	GET on both form endpoints returns the 405 JSON (proves deployment without posting to Slack); BTV feed has videos
Previews	headers on article pages; sitemap lists stories; one story's `og:title` is the story, with JSON-LD
Radio.co	status and history JSON shapes; the stream answers `audio/*`; stream CORS allows the visualizer
Sanity	a count query succeeds on API `v2024-01-01`; CORS allows bearcastmedia.com
iTunes	a known search returns results
Backups	latest `sanity-backup-*` release is at most 9 days old

.GITHUB/SCRIPTS/HEALTH-CHECK.SH · LINES 42-59 HEALTH-CHECK.SH — TELLING BOT PROTECTION APART FROM AN OUTAGE

```
cf_blocked(){
  [ "$CODE" = "403" ] || [ "$CODE" = "503" ] || return 1
  [ -n "$(header cf-mitigated)" ] && return 0
```

```
  printf '%s' "$BODY" | grep -qiE 'challenge-platform|cf-chl|Just a moment|Attention Required! \| Cloudflare' && return 0
  return 1
}

# Which address to test: the real domain, or pages.dev if bot protection
# blocks us (that's Cloudflare working as intended, not an outage).
BASE="$SITE"
fetch "$SITE/"
if cf_blocked; then
  echo "  ℹ️  $SITE blocks automated traffic from this server (Cloudflare bot protection — expected for GitHub's runners)."
  echo "     Checking the same deployment at $PAGES_URL instead."
  BASE="$PAGES_URL"
elif [ "$CODE" = "000" ]; then
  fail "Website" "$SITE didn't respond at all (DNS or Cloudflare problem?)"
fi
```

14.3 Weekly Sanity backup

Every Sunday at 08:00 UTC the dataset is exported twice: a full export with every uploaded image as a GitHub Release (`sanity-backup-YYYY-MM-DD` , up to 2 GB per file), and a small documents-only export on a `backups` branch that is replaced with one fresh commit each week (tagged `[CF-Pages-Skip]` so Cloudflare does not build it). The newest six of each are kept. Backups contain unpublished drafts and contributors' email addresses, so before doing anything the workflow checks the destination repository's visibility and refuses to save to a public one; before the code repository is ever made public, backups move to a separate private repository (`BACKUP_REPO` + a fine-grained token).

.GITHUB/WORKFLOWS/SANITY-BACKUP.YML · LINES 74-90 — SANITY-BACKUP.YML — THE PRIVACY GUARD RUNS FIRST

```
- name: Make sure backups can't be published
  env:
    BACKUP_REPO: ${{ vars.BACKUP_REPO }}
    BACKUP_REPO_TOKEN: ${{ secrets.BACKUP_REPO_TOKEN }}
    GH_TOKEN: ${{ secrets.BACKUP_REPO_TOKEN || secrets.GITHUB_TOKEN }}
  run: |
    set -euo pipefail
    if [ -n "$BACKUP_REPO" ] && [ -z "$BACKUP_REPO_TOKEN" ]; then
      echo "::error::BACKUP_REPO is set to '$BACKUP_REPO' but the BACKUP_REPO_TOKEN secret is missing (fine-grained token with Contents: Read & write on that repo)."
      exit 1
    fi
    private=$(gh api "repos/$TARGET_REPO" --jq .private 2>/dev/null || echo unknown)
    if [ "$private" != "true" ]; then
      echo "::error::Refusing to save backups to $TARGET_REPO — it is not a private repository (visibility check returned '$private'). Backups contain unpublished drafts and personal emails. Set BACKUP_REPO to a private repo (see MAINTENANCE.md)."
      exit 1
    fi
    echo "Backups will be saved to private repo $TARGET_REPO"
```

Restoring is one command from `bearcast-studio/` : `npx sanity dataset import <file>.tar.gz production --replace` — ideally into a test dataset first.

14.4 Dependabot

Dependabot opens at most one grouped pull request a month for GitHub Actions updates. It deliberately does not watch the Studio's npm packages: the Studio is upgraded on purpose a couple of times a year (when two majors behind or for security fixes), following the steps in `MAINTENANCE.md`.

14.5 Running it locally

README.MD · LINE 132 — SERVING THE SITE

```
132  python3 -m http.server 8080      # or: npx serve .   /   VS Code Live Server
```

FEATURE	LOCALLY
Sanity content	Blocked until the local origin is added under API → CORS origins at sanity.io/manage (remove it afterwards). Pages otherwise show their empty/error states.
Song requests, forms	Need the Functions runtime: `npx wrangler pages dev .` with secrets in a git-ignored `.dev.vars`
Asset versions	The literal `__ASSET_VERSION__` works as a version string; never run `build.sh` locally (it refuses)
Radio, visualizer, Apple Music	Work as on the live site
Studio	`cd bearcast-studio && npm install && npm run dev` → localhost:3333

§ 15 — COMPARISON

Before & After — WordPress vs. the Current Build

> *The "before" images are the inherited WordPress site, captured for the original audit in May 2026. The "after" images are rendered from the current source in a sandbox with no network access to Sanity, Radio.co or YouTube — so they show each page's real layout with its loading, empty or idle states rather than live content.*

15.1 Architecture side by side

CONCERN	INHERITED WORDPRESS	CURRENT BUILD
Rendering	PHP theme per request on shared hosting	Static files from Cloudflare's edge; content fetched in the browser
Content model	Posts + categories/tags	Eight purpose-built document types with validation
Admin	wp-admin with plugin notices and upsells	Department-shaped Sanity Studio
Radio	Third-party player plugin	Own player, DJ detection, visualizers, listener count, requests
Forms	Plugin forms	Validated edge functions delivering to private Slack channels
Security headers	Graded F (none of six)	Strict, hash-pinned CSP plus full header set; CI-enforced
Updates	Core, theme, plugins, PHP	No server software; Studio patches auto-apply; Actions via Dependabot
Running cost	Hosting plan	$0 (free tiers)

BEFORE · HOMEPAGE



AFTER · HOMEPAGE

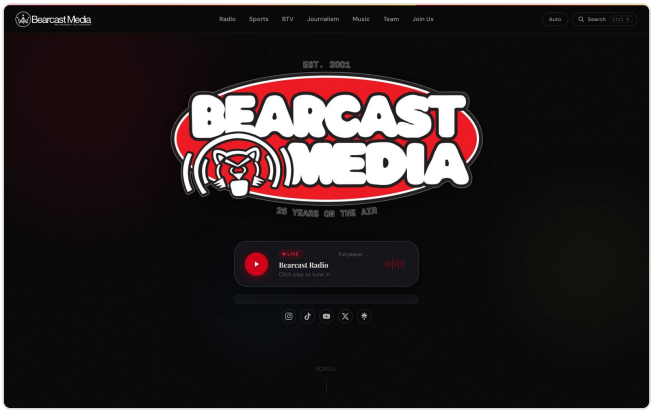
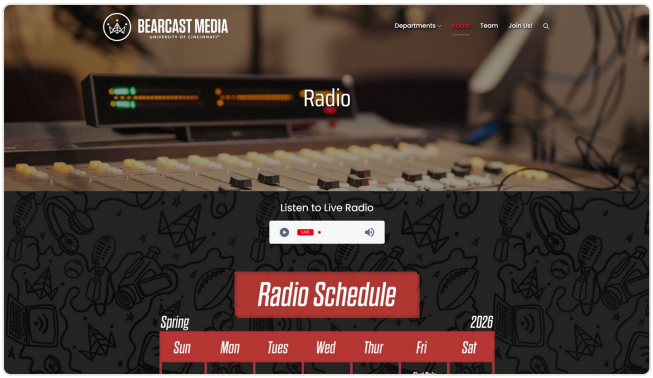


Figure 15.1 — The template's drone-photo masthead and generic crest vs. the 25th-anniversary hero with the radio player front and centre.

BEFORE · RADIO



AFTER · RADIO

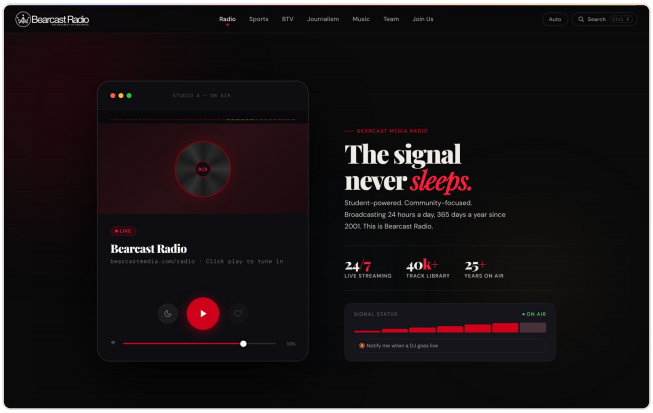
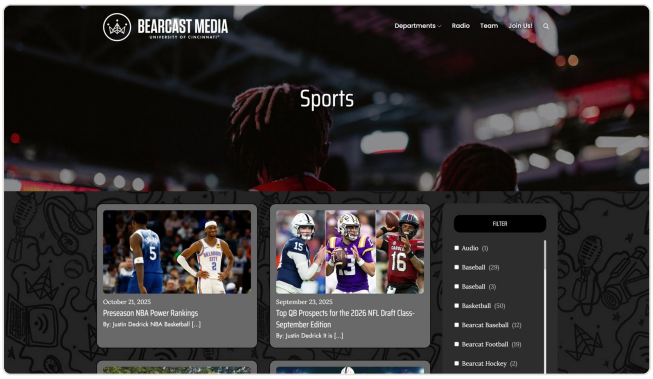


Figure 15.2 — A banner-and-tabs radio page vs. the Studio A player with live signal status.

BEFORE · SPORTS



AFTER · SPORTS

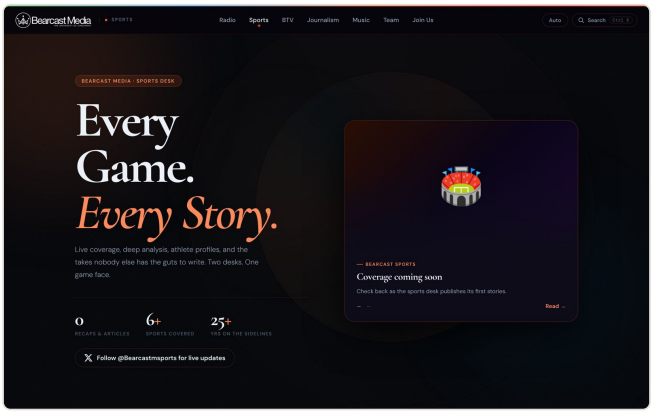
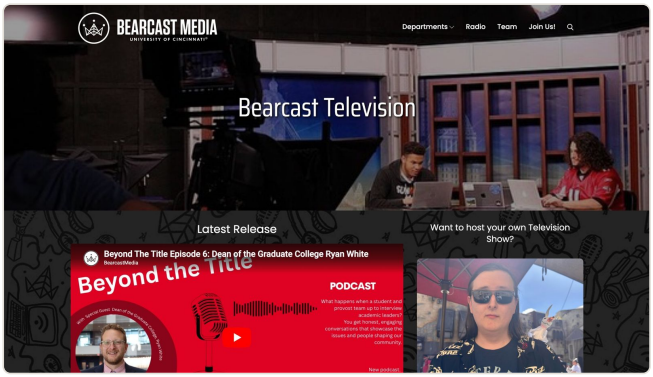


Figure 15.3 — Sports.

BEFORE · BTV



AFTER · BTV

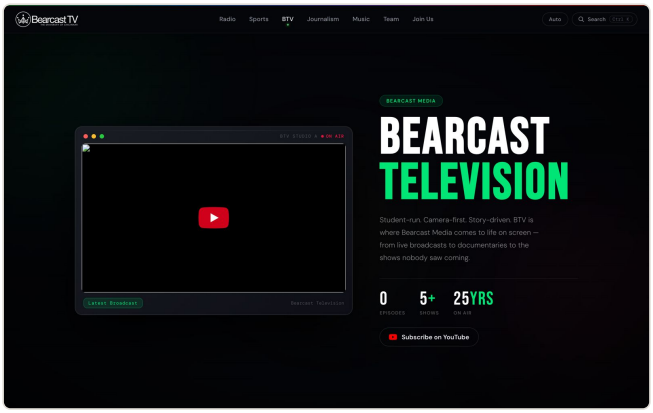
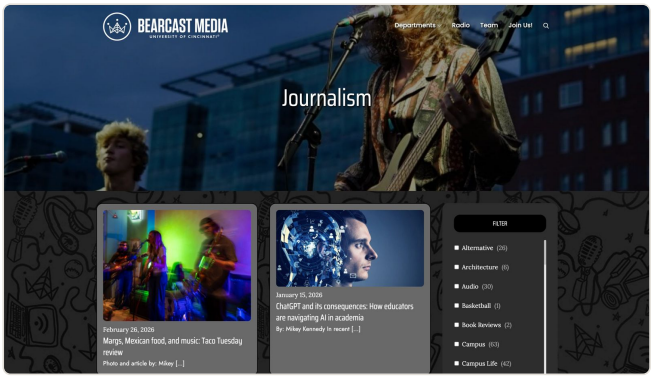


Figure 15.4 — Bearcast Television.

BEFORE · JOURNALISM

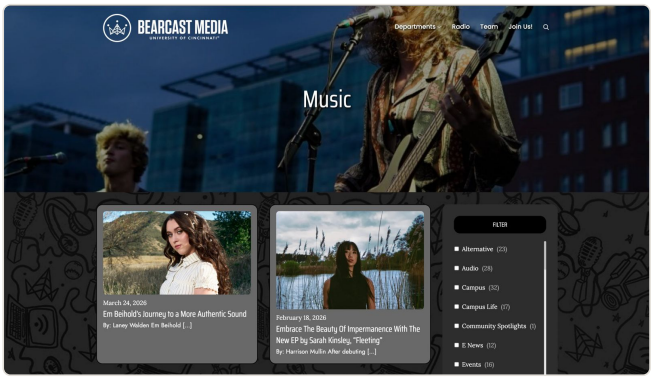


AFTER · JOURNALISM



Figure 15.5 — A generic post grid vs. *The Bearcast* newspaper front.

BEFORE · MUSIC



AFTER · MUSIC

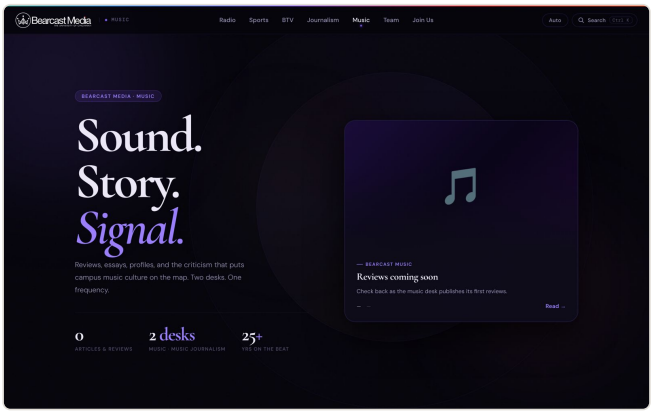


Figure 15.6 — Music.

BEFORE · JOIN



AFTER · JOIN

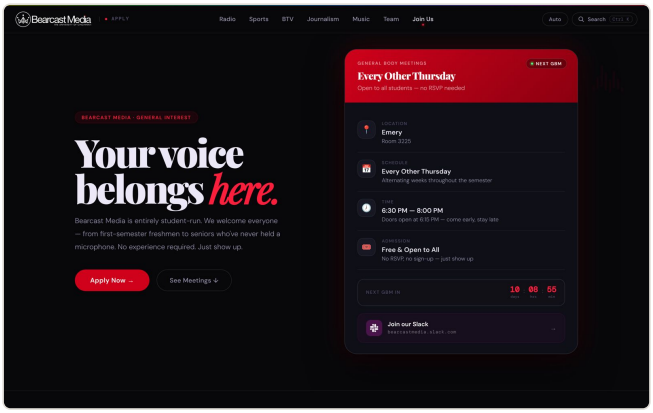


Figure 15.7 — Recruiting.

§ 16 — OPERATIONS

Performance, Operations & Known Limits

16.1 Performance profile

- **First paint from the edge.** HTML, CSS and JS come from the nearest Cloudflare location; repeat visits take CSS/JS from the immutable HTTP cache or the service worker.
- **Smaller pages.** Per-page bundles cut CSS+JS by roughly two-thirds on most pages (§13.2); fonts are self-hosted and only two are preloaded.
- **Less data per page.** Lists ask Sanity for word counts instead of full bodies; bodies load on open; the ⌘K index loads once per session; Site Settings once per 30 days.
- **Quiet background tabs.** `bcInterval` skips polls and animations while a tab is hidden — except the now-playing poll while audio plays, so the lock screen stays current — and runs once on return. Phones draw visualizers at 30 fps.

SCRIPTS.JS · LINES 1891-1900 — SCRIPTS.JS — INTERVALS THAT RESPECT HIDDEN TABS

```
1891  window.bcInterval = function(fn, ms, opts){
1892    var whileHidden = opts && opts.whileHidden;
1893    var id = setInterval(function(){
1894      if (!document.hidden || (whileHidden && whileHidden())) fn();
1895    }, ms);
1896    document.addEventListener('visibilitychange', function(){
1897      if (!document.hidden) fn();
1898    });
1899    return id;
1900  };
```

16.2 Where settings live

SETTING	LOCATION
Radio.co station / stream	`scripts.js` → `BEARCAST_RADIO` (and `health-check.sh`)
Sanity project / dataset / API version	top of `scripts.js`; `sanity.config.ts`; `health-check.sh`; `_middleware.js`
Listener tiers	`BC_TIERS` in `scripts.js` **and** `TIERS` in `song-request.js` — same order
Earliest valid listening day	`BC_HISTORY_START`
Security headers / CSP	`_headers` **and** `SECURITY_HEADERS` in `_middleware.js` (CI keeps them identical)
BTV channel	`CHANNEL_ID` in `btv-videos.js`
Social links	Studio → Site Settings → Social Media (fallback: `BC_SOCIAL.FALLBACK`)
Webhooks, API key, D1	Cloudflare Pages → Settings → Variables and Secrets / Bindings
Backup token / private repo	GitHub → Settings → Secrets and variables → Actions

16.3 Maintenance rhythm

Plan a one-hour check-up twice a year (January and August): open `health-check` issues and Dependabot PRs; confirm recent backup releases and the backup token's expiry; compare the Studio's Sanity major version with the latest and upgrade when two majors behind (validate schema, `tsc --noEmit`, `sanity build`, click through, deploy); spot-check the site on a phone. Between check-ups, react to the automatic alerts.

16.4 What could still go wrong

EVENT	EFFECT	RESPONSE
Sanity down or API version retired	Content pages show "Couldn't load…"; radio unaffected	Health check flags it; bump the API date in the three places
Radio.co changes host or API	Player or now-playing fails	Health check; update `BEARCAST_RADIO`
Stream loses CORS headers	Visualizers fall back to stylized mode; audio plays	Nothing urgent
iTunes API limited	Apple Music buttons stay search links	Nothing urgent
YouTube RSS down / API key invalid	BTV empty or limited to 15 videos	Check `/api/btv-videos` and the key
Slack webhook revoked	Forms report "didn't send"	Recreate webhook, update the secret
Build command changed	Unversioned assets; non-site files published	Health check flags both; restore `bash build.sh`
A bad deploy	Broken page	Cloudflare one-click rollback; fix on a branch

Throttles in the functions live in each Cloudflare isolate's memory, so they stop one person hammering a button, not a distributed flood; a Cloudflare WAF rate-limiting rule is the upgrade path if that is ever needed. Your Bearcast data is per browser and can be lost if a visitor clears site data — export/import is the remedy.

§ 17 — COLOPHON

Colophon, Glossary & Revision History

17.1 The documentation set

Technical White Paper (this document)	What was built and why, with the code.
Cybersecurity White Paper	Threat model, defense layers, findings and the October 2026 verification.
Editor Handbook	How editors use the Studio, field by field.
System Architecture Diagram	Every flow drawn at full size.
`MAINTENANCE.md`, `CONTRIBUTING.md`, `SECURITY.md`	In the repository: the operating runbook, contribution rules and vulnerability reporting.

17.2 Glossary

GROQ	Sanity's query language: filters, projections and joins in one string.
Portable Text	Sanity's JSON format for rich text, rendered by the site's own serializer.
apicdn	Sanity's cached read API (~60 s freshness).
Pages Function	Server code Cloudflare runs at the edge for routes listed in `_routes.json`.
D1	Cloudflare's SQLite database service.
CSP	Content-Security-Policy: the browser-enforced list of what a page may load and run.
Service worker	A script the browser runs between page and network; here, for offline support and caching.
Desk	A value of `article.desk` that decides which page a story appears on.
Hotspot	The focal point an editor sets on an image so crops keep the subject.
Media Session	Browser API that shows now-playing info on lock screens and media keys.

17.3 Revision history

VERSION	DATE	CHANGES
1.0	May 2026	Initial publication.
1.1	Aug 2026	Service worker, shared interaction modules, deployment header files.
2.0	Oct 2026	Full rewrite from the current source with quoted code: own radio player and audio analysis, listener count, Your Bearcast, search engine and reader, BTV from YouTube, Pages Functions (forms, previews, sitemap), build pipeline and per-page bundles, self-hosted fonts, hash-pinned CSP, CI/health/backup automation, content migrations, schema changes (`dept` removed), new screenshots, founding year corrected to 2001.

17.4 Production notes

Set in Playfair Display, DM Sans and DM Mono — the website's own self-hosted faces. Built with WeasyPrint from HTML; every code listing is extracted from the repository snapshot at build time and syntax-highlighted with Pygments, so listings match the code exactly. Screenshots of the current build were rendered with headless Chromium from the same snapshot.

BearcastMedia.com Technical White Paper · v2.0 · October 2026 · Cam Garrison for Bearcast Media · Code © 2026 Cam Garrison (MIT where licensed); Bearcast Media name, logos and content © Bearcast Media.